

Chapter 17. Page Frame Reclaiming

In previous chapters, we explained how the kernel handles dynamic memory by keeping track of free and busy page frames. We have also discussed how every process in User Mode has its own address space and has its requests for memory satisfied by the kernel one page at a time, so that page frames can be assigned to the process at the very last possible moment. Last but not least, we have shown how the kernel makes use of dynamic memory to implement both memory and disk caches .

In this chapter, we complete our description of the virtual memory subsystem by discussing page frame reclaiming. We'll start in the first section, "[The Page Frame Reclaiming Algorithm](#)," explaining why the kernel needs to reclaim page frames and what strategy it uses to achieve this. We then make a technical digression in the section "[Reverse Mapping](#)" to discuss the data structures used by the kernel to locate quickly all the Page Table entries that point to the same page frame. The section "[Implementing the PFRA](#)" is devoted to the page frame reclaiming algorithm used by Linux. The last main section, "[Swapping](#)," is almost a chapter by itself: it covers the swap subsystem, a kernel component used to save anonymous (not mapping data of files) pages on disk.

17.1. The Page Frame Reclaiming Algorithm

One of the fascinating aspects of Linux is that the checks performed before allocating dynamic memory to User Mode processes or to the kernel are somewhat perfunctory.

No rigorous check is made, for instance, on the total amount of RAM assigned to the processes created by a single user (the limits mentioned in the section "[Process Resource Limits](#)" in [Chapter 3](#) mostly affect single processes). Similarly, no limit is placed on the size of the many disk caches and memory caches used by the kernel.

This lack of controls is a design choice that allows the kernel to use the available RAM in the best possible way. When the system load is low, the RAM is filled mostly by the disk caches and the few running processes can benefit from the information stored in them. However, when the system load increases, the RAM is filled mostly by pages of the processes and the caches are shrunk to make room for additional processes.

As we saw in previous chapters, both memory and disk caches grab more and more page frames but never release any of them. This is reasonable because cache systems don't know if and when processes will reuse some of the cached data and are therefore unable to identify the portions of cache that should be released. Moreover, thanks to the demand paging mechanism described in [Chapter 9](#), User Mode processes get page frames as long as they proceed with their execution; however, demand paging has no way to force processes to release the page frames whenever they are no longer used.

Thus, sooner or later all the free memory will be assigned to processes and caches. The page frame reclaiming algorithm of the Linux kernel refills the lists of free blocks of the buddy system by "stealing" page frames from both User Mode processes and kernel caches.

Actually, page frame reclaiming must be performed before all the free memory has been used up. Otherwise, the kernel might be easily trapped in a deadly chain of memory requests that leads to a system crash. Essentially, to free a page frame the kernel must write its data to disk; however, to accomplish this operation, the kernel requires another page frame (for instance, to allocate the buffer heads for the I/O data transfer). If no free page frame exists, no page frame can be freed.

One of the goals of page frame reclaiming is thus to conserve a minimal pool of free page frames so that the kernel may safely recover from "low on memory" conditions.

17.1.1. Selecting a Target Page

The objective of the page frame reclaiming algorithm (PFRA) is to pick up page frames and make them free. Clearly the page frames selected by the PFRA must be non-free, that is, they must not be already included in one of the `free_area` arrays used by the buddy system (see the section "[The Buddy System Algorithm](#)" in [Chapter 8](#)).

The PFRA handles the page frames in different ways, according to their contents. We can distinguish between unreclaimable pages, swappable pages, syncable pages, and discardable pages. These types are explained in [Table 17-1](#).

Table 17-1. The types of pages considered by the PFRA

Type of pages	Description	Reclaim action
Unreclaimable	Free pages (included in buddy system lists)	
	Reserved pages (with <code>PG_reserved</code> flag set)	
	Pages dynamically allocated by the kernel	
	Pages in the Kernel Mode stacks of the processes	(No reclaiming allowed or needed)
Swappable	Temporarily locked pages (with <code>PG_locked</code> flag set)	
	Memory locked pages (in memory regions with <code>VM_LOCKED</code> flag set)	
	Anonymous pages in User Mode address spaces	
	Mapped pages of <code>tmpfs</code> filesystem (e.g., pages of IPC shared memory)	Save the page contents in a swap area
Syncable	Mapped pages in User Mode address spaces	Synchronize the page with its image on disk, if necessary

Discardable	Pages included in the page cache and containing data of disk files	
	Block device buffer pages	
	Pages of some disk caches (e.g., the inode cache)	
	Unused pages included in memory caches (e.g., slab allocator caches)	Nothing to be done
	Unused pages of the dentry cache	

In the above table, a page is said to be mapped if it maps a portion of a file. For instance, all pages in the User Mode address spaces belonging to file memory mappings are mapped, as well as any other page included in the page cache. In almost all cases, mapped pages are syncable: in order to reclaim the page frame, the kernel must check whether the page is dirty and, if necessary, write the page contents in the corresponding disk file.

Conversely, a page is said to be anonymous if it belongs to an anonymous memory region of a process (for instance, all pages in the User Mode heap or stack of a process are anonymous). In order to reclaim the page frame, the kernel must save the page contents in a dedicated disk partition or disk file called "swap area" (see the later section "[Swapping](#)"); therefore, all anonymous pages are swappable.

Usually, the pages of special filesystems are not reclaimable. The only exceptions are the pages of the tmpfs special filesystem, which can be reclaimed by saving them in a swap area. As we'll see in [Chapter 19](#), the tmpfs special filesystem is used by the IPC shared memory mechanism.

When the PFRA must reclaim a page frame belonging to the User Mode address space of a process, it must take into consideration whether the page frame is shared or non-shared . A shared page frame belongs to multiple User Mode address spaces, while a non-shared page frame belongs to just one. Notice that a non-shared page frame might belong to several lightweight processes referring to the same memory descriptor.

Shared page frames are typically created when a process spawns a child; as explained in the section "[Copy On Write](#)" in [Chapter 9](#), the page tables of the child are copied from those of the parent, thus parent and child share the same page frames. Another common case occurs when two or more processes access the same file by means of a shared memory mapping (see the section "[Memory Mapping](#)" in [Chapter 16](#)).^[*]

[*] It should be noted, however, that when a single process accesses a file through a shared memory mapping, the corresponding pages are non-shared as far as the PFRA is concerned. Similarly, a page belonging to a private memory mapping may be treated as shared by the PFRA (for instance, because two processes read the same file portion and none of them modified the data in the page).

17.1.2. Design of the PFRA

While it is easy to identify the page candidates for memory reclaiming roughly speaking, any page belonging to a disk or memory cache, or to the User Mode address space of a process selecting the proper target pages is perhaps the most sensitive issue in kernel design.

As a matter of fact, the hardest job of a developer working on the virtual memory subsystem consists of finding an algorithm that ensures acceptable performance both for desktop machines (on which memory requests are quite limited but system responsiveness is crucial) and for high-level machines such as large database servers (on which memory requests tend to be huge).

Unfortunately, finding a good page frame reclaiming algorithm is a rather empirical job, with very little support from theory. The situation is somewhat similar to evaluating the factors that determine the dynamic priority of a process: the main objective is to tune the parameters in such a way to achieve good system performance, without asking too many questions about why it works well. Often, it's just a matter of "let's try this approach and see what happens." An unpleasant side effect of this empirical design is that the code changes quickly. For that reason, we cannot ensure that the memory reclaiming algorithm we are going to describe the one used in Linux 2.6.11 will be exactly the same, by the time you'll read this chapter, as the one adopted by the most up-to-date version of the Linux 2.6 kernel. However, the general ideas and the main heuristic rules described here should continue to hold.

Looking too close to the trees' leaves might lead us to miss the whole forest. Therefore, let us present a few general rules adopted by the PFRA. These rules are embedded in the functions that will be described later in this chapter.

Free the "harmless" pages first

Pages included in disk and memory caches not referenced by any process should be reclaimed before pages belonging to the User Mode address spaces of the processes; in the former case, in fact, the page frame reclaiming can be done without modifying any Page Table entry. As we will see in the section "[The Least Recently Used \(LRU\) Lists](#)" later in this chapter, this rule is somewhat mitigated by introducing a "swap tendency factor."

Make all pages of a User Mode process reclaimable

With the exception of locked pages, the PFRA must be able to steal any page of a User Mode process, including the anonymous pages. In this way, processes that have been sleeping for a long period of time will progressively lose all their page frames.

Reclaim a shared page frame by unmapping at once all page table entries that reference it

When the PFRA wants to free a page frame shared by several processes, it clears all page table entries that refer to the shared page frame, and then reclaims the page frame.

Reclaim "unused" pages only

The PFRA uses a simplified Least Recently Used (LRU) replacement algorithm to classify pages as in-use and unused.^[*] If a page has not been accessed for a long time, the probability that it will be accessed in the near future is low and it can be considered "unused;" on the other hand, if a page has been accessed recently, the probability that it will continue to be accessed is high and it must be considered as "in-use." The PFRA reclaims only unused pages. This is just another application of the locality principle mentioned in the section "[Hardware Cache](#)" in [Chapter 2](#).

[*] The PFRA could also be considered as a "used-once" algorithm, which has its roots in the 2Q buffer management replacement algorithm proposed by T. Johnson and D. Shasha in 1994.

The main idea behind the LRU algorithm is to associate a counter storing the age of the page with each page in RAM that is, the interval of time elapsed since the last access to the page. This counter allows the PFRA to reclaim only the oldest page of any process. Some computer platforms provide sophisticated support for LRU algorithms;^[†] unfortunately, 80 x 86 processors do not offer such a hardware feature, thus the Linux kernel cannot rely on a page counter that keeps track of the age of every page. To cope with this restriction, Linux takes advantage of the `Accessed` bit included in each Page Table entry, which is automatically set by the hardware when the page is accessed; moreover, the age of a page is represented by the position of the page descriptor in one of two different lists (see the section "[The Least Recently Used \(LRU\) Lists](#)" later in this chapter).

[†] For instance, the CPUs of some mainframes automatically update the value of a counter included in each page table entry to specify the age of the corresponding page.

Therefore, the page frame reclaiming algorithm is a blend of several heuristics:

- Careful selection of the order in which caches are examined.
- Ordering of pages based on aging (least recently used pages should be freed before pages accessed recently).
- Distinction of pages based on the page state (for example, non-dirty pages are better candidates than dirty pages because they don't have to be written to disk).

17.2. Reverse Mapping

As stated in the previous section, one of the objectives of the PFRA is to be able to free a shared page frame. To that end, the Linux 2.6 kernel is able to locate quickly all the Page Table entries that point to the same page frame. This activity is called reverse mapping .

A trivial solution for reverse mapping would be to include in each page descriptor additional fields to link together all the Page Table entries that point to the page frame associated with the page descriptor. However, keeping such lists up-to-date would increase significantly the kernel overhead; for that reason, more sophisticated solutions have been devised. The technique used in Linux 2.6 is named object-based reverse mapping. Essentially, for any reclaimable User Mode page, the kernel stores the backward links to all memory regions in the system (the "objects") that include the page itself. Each memory region descriptor stores a pointer to a memory descriptor, which in turn includes a pointer to a Page Global Directory. Therefore, the backward links enable the PFRA to retrieve all Page Table entries referencing a given a page. Because there are fewer memory region descriptors than page descriptors, updating the backward links of a shared page is less time consuming. Let's see how this scheme is worked out.

First of all, the PFRA must have a way to determine whether the page to be reclaimed is shared or non-shared, and whether it is mapped or anonymous. In order to do this, the kernel looks at two fields of the page descriptor: `_mapcount` and `mapping`.

The `_mapcount` field stores the number of Page Table entries that refer to the page frame. The counter starts from `-1`: this value means that no Page Table entry references the page frame. Thus, if the counter is zero, the page is non-shared, while if it is greater than zero the page is shared. The `page_mapcount ( )` function receives the address of a page descriptor and returns the value of its `_mapcount` plus one (thus, for instance, it returns one for a non-shared page included in the User Mode address space of some process).

The `mapping` field of the page descriptor determines whether the page is mapped or anonymous, as follows:

- If the `mapping` field is `NULL`, the page belongs to the swap cache (see the section "[The Swap Cache](#)" later in this chapter).
- If the `mapping` field is not `NULL` and its least significant bit is 1, it means the page is anonymous and the `mapping` field encodes the pointer to an `anon_vma` descriptor (see the next section, "[Reverse Mapping for Anonymous Pages](#)").
- If the `mapping` field is non-`NULL` and its least significant bit is 0, the page is mapped; the `mapping` field points to the `address_space` object of the corresponding file (see the section "[The address_space Object](#)" in [Chapter 15](#)).

Every `address_space` object used by Linux is aligned in RAM so that its starting linear address is a multiple of four. Therefore, the least significant bit of the `mapping` field can be used as a flag denoting whether the field contains a pointer to an `address_space` object or to an `anon_vma` descriptor. This is a dirty programming trick, but the kernel uses a lot of page descriptors, thus these data structures should be as small as possible. The `PageAnon ( )` function receives as its parameter the address of a page descriptor and returns 1 if the least significant bit of the `mapping` field is set, 0 otherwise.

The `TRy_to_unmap ( )` function, which receives as its parameter a pointer to a page descriptor, tries to clear all the Page Table entries that point to the page frame associated with that page descriptor. The function returns `SWAP_SUCCESS` (zero) if the function succeeded in removing any reference to the page frame from all Page Table entries, it returns `SWAP_AGAIN` (one) if some reference could not be removed, and returns `SWAP_FAIL` (two) in case of errors. The function is quite short:

```

int try_to_unmap(struct page *page)
{
    int ret;
    if (PageAnon(page))
        ret = try_to_unmap_anon(page);
    else
        ret = try_to_unmap_file(page);
    if (!page_mapped(page))
        ret = SWAP_SUCCESS;
    return ret;
}

```

The `try_to_unmap_anon( )` and `try_to_unmap_file( )` functions take care of anonymous pages and mapped pages, respectively. These functions will be described in the forthcoming sections.

17.2.1. Reverse Mapping for Anonymous Pages

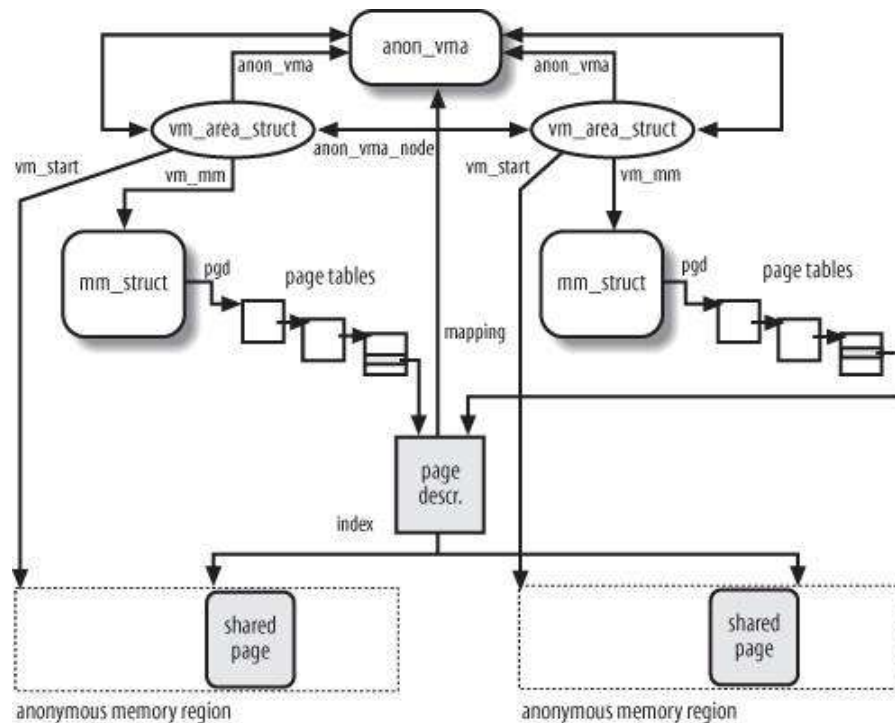
Anonymous pages are often shared among several processes. The most common case occurs when forking a new process: as explained in the section "Copy On Write" in [Chapter 9](#), all page frames owned by the parent including the anonymous pages are assigned also to the child. Another (quite unusual) case occurs when a process creates a memory region specifying both the `MAP_ANONYMOUS` and `MAP_SHARED` flag: the pages of such a region will be shared among the future descendants of the process.

The strategy to link together all the anonymous pages that refer to the same page frame is simple: the anonymous memory regions that include the page frame are collected in a doubly linked circular list. Be warned that, even if an anonymous memory region includes different pages, there always is just one reverse mapping list for all the page frames in the region.

When the kernel assigns the first page frame to an anonymous region, it creates a new `anon_vma` data structure, which includes just two fields: `lock`, a spin lock for protecting the list against race conditions, and `head`, the head of the doubly linked circular list of memory region descriptors. Then, the kernel inserts the `vm_area_struct` descriptor of the anonymous memory region in the `anon_vma`'s list; to that end, the `vm_area_struct` data structure includes two fields related to this list: `anon_vma_node` stores the pointers to the next and previous elements in the list, while `anon_vma` points to the `anon_vma` data structure. Finally, the kernel stores the address of the `anon_vma` data structure in the `mapping` field of the descriptor of the anonymous page, as described previously. See [Figure 17-1](#).

When a page frame already referenced by one process is inserted into a Page Table entry of another process (for instance, as a consequence of a `fork( )` system call, see

Figure 17-1. Object-based reverse mapping for anonymous pages



the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#)); the kernel simply inserts the anonymous memory region of the second process in the doubly linked circular list of the `anon_vma` data structure pointed to by the `anon_vma` field of the first process's memory region. Therefore, any `anon_vma`'s list typically includes memory regions owned by different processes.^[*]

[*] An `anon_vma`'s list may also include several adjacent anonymous memory regions owned by the same process. Usually this occurs when an anonymous memory region is split in two or more regions by the `mprotect( )` system call.

As shown in [Figure 17-1](#), the `anon_vma`'s list allows the kernel to quickly locate all Page Table entries that refer to the same anonymous page frame. In fact, each region descriptor stores in the `vm_mm` field the address of the memory descriptor, which in turn includes a field `pgd` containing the address of the Page Global Directory of the process. The Page Table entry can then be determined by considering the starting linear address of the anonymous page, which is easily obtained from the memory region descriptor and the `index` field of the page descriptor.

17.2.1.1. The `try_to_unmap_anon( )` function

When reclaiming an anonymous page frame, the PFRA must scan all memory regions in the `anon_vma`'s list and carefully check whether each region actually includes an anonymous page whose underlying page frame is the target page frame. This job is done by the `try_to_unmap_anon( )` function, which receives as its parameter the descriptor of the target page frame and performs essentially the following steps:

1. Acquires the lock spin lock of the `anon_vma` data structure pointed to by the `mapping` field of the page descriptor.
2. Scans the `anon_vma`'s list of memory region descriptors; for each `vma` memory region descriptor found in that list, it invokes the `try_to_unmap_one( )` function passing as parameters `vma` and the page descriptor (see below). If for some reason this function returns a `SWAP_FAIL` value, or if the `_mapcount` field of the page descriptor indicates that all Page Table entries referencing the page

- frame have been found, the scanning terminates before reaching the end of the list.
- 3. Releases the spin lock obtained in step 1.
- 4. Returns the value computed by the last invocation of `TRY_to_unmap_one( )`: `SWAP_AGAIN` (partial success) or `SWAP_FAIL` (failure).

17.2.1.2. The `try_to_unmap_one( )` function

The `TRY_to_unmap_one( )` function is called repeatedly both from `try_to_unmap_anon( )` and from `TRY_to_unmap_file( )`. It acts on two parameters: a pointer `page` to a target page descriptor and a pointer `vma` to a memory region descriptor. The function essentially performs the following actions:

1. Computes the linear address of the page to be reclaimed from the starting linear address of the memory region (`vma->vm_start`), the offset of the memory region in the mapped file (`vma->vm_pgoff`), and the offset of the page inside the mapped file (`page->index`). For anonymous pages, the `vma->vm_pgoff` field is either zero or equal to `vm_start/PAGE_SIZE`; correspondingly, the `page->index` field is either the index of the page inside the region or the linear address of the page divided by `PAGE_SIZE`.
2. If the target page is anonymous, it checks whether its linear address falls inside the memory region; if not, it terminates by returning `SWAP_AGAIN`. (As explained when introducing reverse mapping for anonymous pages, the `anon_vma`'s list may include memory regions that do not contain the target page.)
3. Gets the address of the memory descriptor from `vma->vm_mm`, and acquires the `vma->vm_mm->page_table_lock` spin lock that protects the page tables.
4. Invokes successively `pgd_offset( )`, `pud_offset( )`, `pmd_offset( )`, and `pte_offset_map( )` to get the address of the Page Table entry that corresponds to the linear address of the target page.
5. Performs a few checks to verify that the target page is effectively reclaimable. If any of the following checks fails, the function jumps to step 12 to terminate by returning a proper error number, either `SWAP_AGAIN` or `SWAP_FAIL`:
 - a. Checks that the Page Table entry points to the target page; if not, the function returns `SWAP_AGAIN`. This can happen in the following cases:
 - ◊ The Page Table entry refers to a page frame assigned with COW, but the anonymous memory region identified by `vma` still belongs to the `anon_vma` list of the original page frame.
 - ◊ The `mremap( )` system call may remap memory regions and move the pages into the User Mode address space by directly modifying the page table entries. In this particular case, object-based reverse mapping does not work, because the `index` field of the page descriptor cannot be used to determine the actual linear address of the page.
 - ◊ The file memory mapping is non-linear (see the section "[Non-Linear Memory Mappings](#)" in [Chapter 16](#)).
 - b. Checks that the memory region is not locked (`VM_LOCKED`) or reserved (`VM_RESERVED`); if one of these restrictions is in place, the function returns `SWAP_FAIL`.
 - c. Checks that the `Accessed` bit inside the Page Table entry is cleared; if not, the function clears the bit and returns `SWAP_FAIL`. If the `Accessed` bit is set, the page is considered in-use, thus it should not be reclaimed.
 - d. Checks whether the page belongs to the swap cache (see the section "[The Swap Cache](#)" later in this chapter) and it is currently being handled by `get_user_pages( )` (see the section "[Allocating a Linear Address Interval](#)" in [Chapter 9](#)); in this case, to avoid a nasty race condition, the function returns `SWAP_FAIL`.

6. The page can be reclaimed: if the `Dirty` bit in the Page Table entry is set, sets the `PG_dirty` flag of the page.
7. Clears the Page Table entry and flushes the corresponding TLBs.
8. If the page is anonymous, the function inserts a swapped-out page identifier in the Page Table entry so that further accesses to this page will swap in the page (see the section "[Swapping](#)" later in this chapter). Moreover, it decreases the counter of anonymous pages stored in the `anon_rss` field of the memory descriptor.
9. Decreases the counter of page frames allocated to the process stored in the `rss` field of the memory descriptor.
10. Decreases the `_mapcount` field of the page descriptor, because a reference to this page frame in the User Mode Page Table entries has been deleted.
11. Decreases the usage counter of the page frame, which is stored in the `_count` field of the page descriptor. If the counter becomes negative, it removes the page descriptor from the active or inactive list (see the section "[The Least Recently Used \(LRU\) Lists](#)" later in this chapter), and invokes `free_hot_page( )` to release the page frame (see the section "[The Per-CPU Page Frame Cache](#)" in [Chapter 8](#)).
12. Invokes `pte_unmap( )` to release the temporary kernel mapping that could have been allocated by `pte_offset_map( )` in step 4 (see the section "[Kernel Mappings of High-Memory Page Frames](#)" in [Chapter 8](#)).
13. Releases the `vma->vm_mm->page_table_lock` spin lock acquired in step 3.
14. Returns the proper error code (`SWAP_AGAIN` in case of success).

17.2.2. Reverse Mapping for Mapped Pages

As with anonymous pages, object-based reverse mapping for mapped pages is based on a simple idea: it is always possible to retrieve the Page Table entries that refer to a given page frame by accessing the descriptors of the memory regions that include the corresponding mapped pages. Thus, the core of reverse mapping is a clever data structure that collects all memory region descriptors relative to a given page frame.

We have seen in the previous section that descriptors for anonymous memory regions are collected in doubly linked circular lists; retrieving all page table entries referencing a given page frame involves a linear scanning of the elements in the list. The number of shared anonymous page frames is never very large, hence this approach works well.

Contrary to anonymous pages, mapped pages are frequently shared, because many different processes may share the same pages of code. For instance, consider that nearly all processes in the system share the pages containing the code of the standard C library (see the section "[Libraries](#)" in [Chapter 20](#)). For this reason, Linux 2.6 relies on special search trees, called "priority search trees," to quickly locate all the memory regions that refer to the same page frame.

There is a priority search tree for every file; its root is stored in the `i_mmap` field of the `address_space` object embedded in the file's `inode` object. It is always possible to quickly retrieve the root of the search tree, because the `mapping` field in the descriptor of a mapped page points to the `address_space` object.

17.2.2.1. The priority search tree

The priority search tree (PST) used by Linux 2.6 is based on a data structure introduced by Edward McCreight in 1985 to represent a set of overlapping intervals. McCreight's tree is a hybrid of a heap and a balanced search tree, and it is used to perform queries on the set of intervals: e.g., "what intervals are contained in a given interval?" and "what intervals intersect a given interval?" in an amount of time directly proportional to

the height of the tree and the number of intervals in the answer.

Each interval in a PST corresponds to a node of the tree, and it is characterized by two indices: the radix index, which corresponds to the starting point of the interval, and the heap index, which corresponds to the final point. The PST is essentially a search tree on the radix index, with the additional heap-like property that the heap index of a node is never smaller than the heap indices of its children.

The Linux priority search tree differs from McCreight's data structure in two important aspects: first, the Linux tree is not always kept balanced (the balancing algorithm is costly both in memory space and in execution time); second, the Linux tree is adapted so as to store memory regions instead of linear intervals.

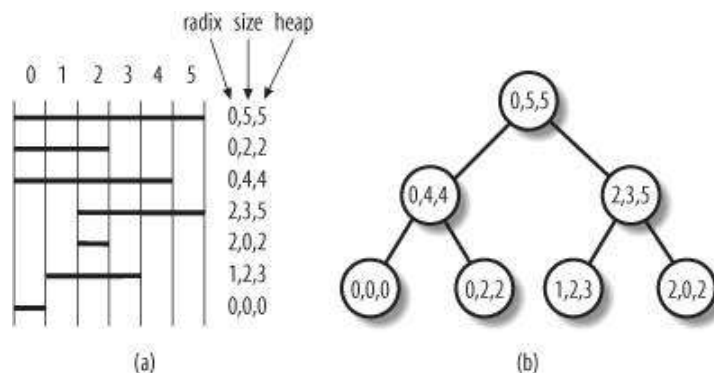
Each memory region can be considered as an interval of file pages identified by the initial position in the file (the radix index) and the final position (the heap index). However, memory regions tend to start from the same pages (typically, from page index 0). Unfortunately, McCreight's original data structure cannot store intervals having the very same starting point. As a partial solution, each node of a PST carries an additional size index other than the radix and heap indices corresponding to the size of the memory region in pages minus one. The size index allows the search program to distinguish different memory regions that start at the same file position.

The size index, however, increases significantly the number of different nodes that may end up in a PST. In particular, if there are too many nodes having the same radix index but different heap indices, the PST could not contain all of them. To solve this problem, the PST may include overflow subtrees rooted at the leaves of the PST and containing nodes having a common radix tree.

Furthermore, different processes may own memory regions that map exactly the same portion of the same file (just consider the example of the standard C library mentioned above). In that case, all nodes corresponding to these memory regions have the same radix, heap, and size indices. When the kernel must insert in a PST a memory region having the same indices as the ones of a node already existing, it inserts the memory region descriptor in a doubly linked circular list rooted at the older PST node.

Figure 17-2 shows a simple example of priority search tree. In the left side of the figure, we show seven memory regions covering the first six pages of a file; each interval is labeled with the radix index, size index, and heap index. In the right side of the figure, we draw the corresponding PST. Notice that no child node has a heap index greater than the heap index of the parent. Also observe that the radix index of the left child of any node is never greater than the radix index of the right child; in case of tie between the radix indices, the ordering is given by the size index. Let us suppose that the PFRA must retrieve all memory regions that include the page at index five. The search algorithm starts at the root (0,5,5): because the corresponding interval includes the page, this is the first retrieved memory region. Then the algorithm visits the left child (0,4,4) of the root and compares the heap index (four) with the page index: because the heap index is smaller, the interval does not include the page; moreover, thanks to the heap-like property of the PST, none of the children of this node can include the page. Thus the algorithm directly jumps to the right child (2,3,5) of the root. The corresponding interval includes the page, hence it is retrieved. Then the algorithm visits the children (1,2,3) and (2,0,2), but it discovers that neither of them include the page.

Figure 17-2. A simple example of priority search tree



We won't be able, for lack of space, to describe in detail the data structures and the functions that implement the Linux PSTs. We'll only mention that a node of a PST is represented by a `prio_tree_node` data structure, which is embedded in the `shared.prio_tree_node` field of each memory region descriptor. The `shared.vm_set` data structure is used as an alternative to `shared.prio_tree_node` to insert the memory region descriptor in a duplicate list of a PST node. PST nodes can be inserted and removed by executing the `vma_prio_tree_insert()` and `vma_prio_tree_remove()` functions; both of them receive as their parameters the address of a memory region descriptor and the address of a PST root. Queries on the PST can be performed by executing the `vma_prio_tree_foreach` macro, which implements a loop over all memory region descriptors that includes at least one page in a specified range of linear addresses.

17.2.2.2. The `try_to_unmap_file()` function

The `try_to_unmap_file()` function is invoked by `TRY_to_unmap()` to perform the reverse mapping of mapped pages. This function is quite simple to describe when the memory mapping is linear (see the section "[Memory Mapping](#)" in [Chapter 16](#)). In this case, it performs the following actions:

1. Gets the `page->mapping->i_mmap_lock` spin lock.
2. Applies the `vma_prio_tree_foreach()` macro to the priority search tree whose root is stored in the `page->mapping->i_mmap` field. For each `vm_area_struct` descriptor found by the macro, the function invokes `try_to_unmap_one()` to try to clear the Page Table entry of the memory region that contains the page (see the earlier section "[Reverse Mapping for Anonymous Pages](#)"). If for some reason this function returns a `SWAP_FAIL` value, or if the `_mapcount` field of the page descriptor indicates that all Page Table entries referencing the page frame have been found, the scanning terminates immediately.
3. Releases the `page->mapping->i_mmap_lock` spin lock.
4. Returns either `SWAP_AGAIN` or `SWAP_FAIL` according to whether all page table entries have been cleared.

If the mapping is non-linear (see the section "[Non-Linear Memory Mappings](#)" in [Chapter 16](#)), the `try_to_unmap_one()` function may fail to clear some Page Table entries, because the `index` field of the page descriptor, which as usual stores the position of the page in the file, is no longer related to the position of the page in the memory region. Therefore, `try_to_unmap_one()` cannot determine the linear address of the page, hence it cannot get the Page Table entry address.

The only solution is an exhaustive search in all the non-linear memory regions of the file. The doubly linked list rooted at the `i_mmap_nonlinear` field of the `page->mapping` file's `address_space` object includes the descriptors of all non-linear memory regions of the file. For each such memory region,

`try_to_unmap_file( )` invokes the `try_to_unmap_cluster( )` function, which scans all Page Table entries corresponding to the linear addresses of the memory region and tries to clear them.

Because the search might be quite time-consuming, a limited scan is performed and a heuristic rule determines the portion of the memory region to be scanned: the `vm_private_data` field of the `vma_area_struct` descriptor holds the current cursor in the current scan. This means that `try_to_unmap_file( )` might in some cases end up missing the page to be unmapped. When this occurs, `try_to_unmap( )` discovers that the page is still mapped and return `SWAP_AGAIN` instead of `SWAP_SUCCESS`.

17.3. Implementing the PFRA

The page frame reclaiming algorithm must take care of many kinds of pages owned by User Mode processes, disk caches and memory caches; moreover, it has to obey several heuristic rules. Thus, it is not surprising that the PFRA is composed of a large number of functions. [Figure 17-3](#) shows the main PFRA functions; an arrow denotes a function invocation, thus for instance `try_to_free_pages( )` invokes `shrink_caches( )`, `shrink_slab( )`, and `out_of_memory( )`.

As you can see, there are several "entry points" for the PFRA. Actually, page frame reclaiming is performed on essentially three occasions:

Low on memory reclaiming

The kernel detects a "low on memory" condition.

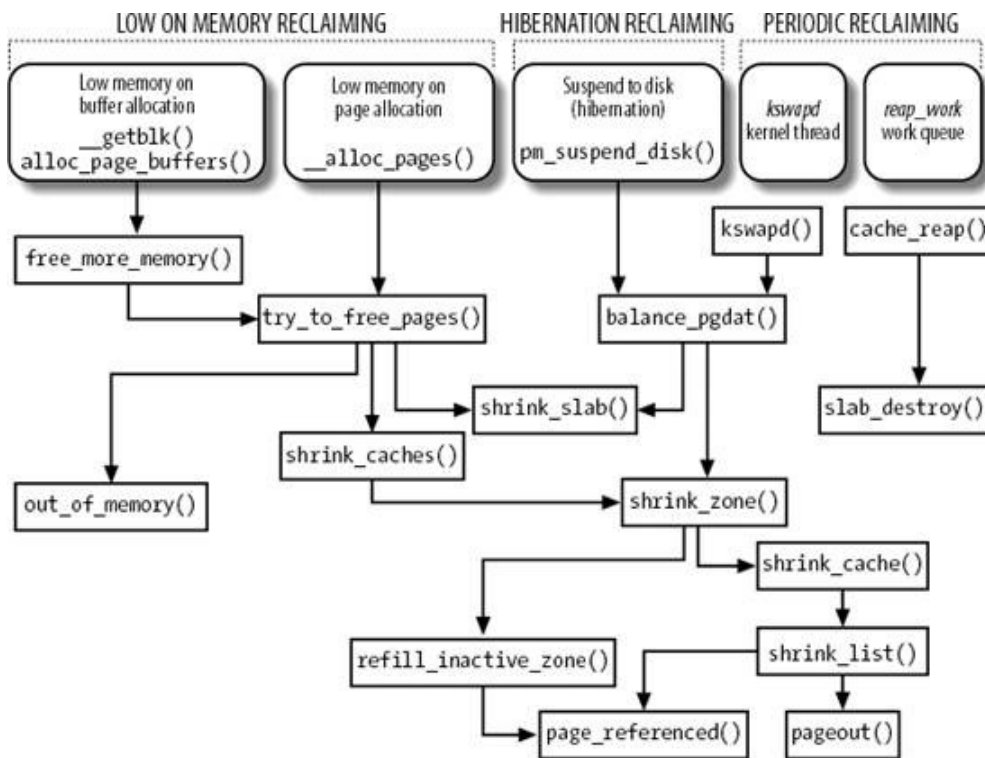
Hibernation reclaiming

The kernel must free memory because it is entering in the suspend-to-disk state (we don't further discuss this case).

Periodic reclaiming

A kernel thread is activated periodically to perform memory reclaiming, if necessary.

Figure 17-3. The main functions of the PFRA



Low on memory reclaiming is activated in the following cases:

- The `grow_buffers()` function, invoked by `__getblk()`, fails to allocate a new buffer page (see the section "[Searching Blocks in the Page Cache](#)" in [Chapter 15](#)).
- The `alloc_page_buffers()` function, invoked by `create_empty_buffers()`, fails to allocate the temporary buffer heads for a page (see the section "[Reading and Writing a File](#)" in [Chapter 16](#)).
- The `_alloc_pages()` function fails in allocating a group of contiguous page frames in a given list of memory zones (see the section "[The Buddy System Algorithm](#)" in [Chapter 8](#)).

Periodic reclaiming is activated by two different types of kernel threads:

- The `kswapd` kernel threads, which check whether the number of free page frames in some memory zone has fallen below the `pages_high` watermark (see the later section "[Periodic Reclaiming](#)").
- The events kernel threads, which are the worker threads of the predefined work queue (see the section "[Work Queues](#)" in [Chapter 4](#)); the PFRA periodically schedules the execution of a task in the predefined work queue to reclaim all free slabs included in the memory caches handled by the slab allocator (see the section "[The Slab Allocator](#)" in [Chapter 8](#)).

We are now going to discuss in detail the various components of the page frame reclaiming algorithm, including all functions shown in [Figure 17-3](#).

17.3.1. The Least Recently Used (LRU) Lists

All pages belonging to the User Mode address space of processes or to the page cache are grouped into two lists called the active list and the inactive list ; they are also collectively denoted as LRU lists . The former list tends to include the pages that have been accessed recently, while the latter tends to include the pages that have not been accessed for some time. Clearly, pages should be stolen from the inactive list.

The active list and the inactive list of pages are the core data structures of the page frame reclaiming algorithm. The heads of these two doubly linked lists are stored, respectively, in the `active_list` and `inactive_list` fields of each zone descriptor (see the section "[Memory Zones](#)" in [Chapter 8](#)). The `nr_active` and `nr_inactive` fields in the same descriptor store the number of pages in the two lists. Finally, the `lru_lock` field is a spin lock that protects the two lists against concurrent accesses in SMP systems.

If a page belongs to an LRU list, its `PG_lru` flag in the page descriptor is set. Moreover, if the page belongs to the active list, the `PG_active` flag is set, while if it belongs to the inactive list, the `PG_active` flag is cleared. The `lru` field of the page descriptor stores the pointers to the next and previous elements in the LRU list.

Several auxiliary functions are available to handle the LRU lists:

`add_page_to_active_list( )`

Adds the page to the head of the zone's active list and increases the `nr_active` field of the zone descriptor.

`add_page_to_inactive_list( )`

Adds the page to the head of the zone's inactive list and increases the `nr_inactive` field of the zone descriptor.

`del_page_from_active_list( )`

Removes the page from the zone's active list and decreases the `nr_active` field of the zone descriptor.

`del_page_from_inactive_list( )`

Removes the page from the zone's inactive list and decreases the `nr_inactive` field of the zone descriptor.

`del_page_from_lru( )`

Checks the `PG_active` flag of a page; according to the result, removes the page from the active or inactive list, decreases the `nr_active` or `nr_inactive` field of the zone descriptor, and clears, if necessary, the `PG_active` flag.

`activate_page( )`

Checks the `PG_active` flag; if it is clear (the page is in the inactive list), it

moves the page into the active list: invokes `del_page_from_inactive_list()`, then invokes `add_page_to_active_list()`, and finally sets the `PG_active` flag. The zone's `lru_lock` spin lock is acquired before moving the page.

`lru_cache_add()`

If the page is not included in an LRU list, it sets the `PG_lru` flag, acquires the zone's `lru_lock` spin lock, and invokes `add_page_to_inactive_list()` to insert the page in the zone's inactive list.

`lru_cache_add_active()`

If the page is not included in an LRU list, it sets the `PG_lru` and `PG_active` flags, acquires the zone's `lru_lock` spin lock, and invokes `add_page_to_active_list()` to insert the page in the zone's active list.

Actually, the last two functions, `lru_cache_add()` and `lru_cache_add_active()`, are slightly more complicated. In fact, the two functions do not immediately move the page into an LRU; instead, they accumulate the pages in temporary data structures of type `pagevec`, each of which may contain up to 14 page descriptor pointers. The pages will be effectively moved in an LRU list only when a `pagevec` structure is completely filled. This mechanism enhances the system performance, because the LRU spin lock is acquired only when the LRU lists are effectively modified.

17.3.1.1. Moving pages across the LRU lists

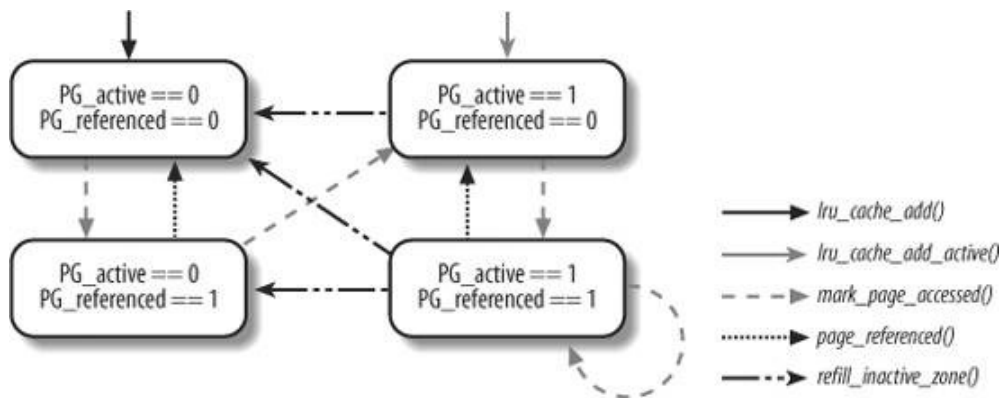
The PFRA collects the pages that were recently accessed in the active list so that it will not scan them when looking for a page frame to reclaim. Conversely, the PFRA collects the pages that have not been accessed for a long time in the inactive list. Of course, pages should move from the inactive list to the active list and back, according to whether they are being accessed.

Clearly, two page states ("active" and "inactive") are not sufficient to describe all possible access patterns. For instance, suppose a logger process writes some data in a page once every hour. Although the page is "inactive" for most of the time, the access makes it "active," thus denying the reclaiming of the corresponding page frame, even if it is not going to be accessed for an entire hour. Of course, there is no general solution to this problem, because the PFRA has no way to predict the behavior of User Mode processes; however, it seems reasonable that pages should not change their status on every single access.

The `PG_referenced` flag in the page descriptor is used to double the number of accesses required to move a page from the inactive list to the active list; it is also used to double the number of "missing accesses" required to move a page from the active list to the inactive list (see below). For instance, suppose that a page in the inactive list has the `PG_referenced` flag set to 0. The first page access sets the value of the flag to 1, but the page remains in the inactive list. The second page access finds the flag set and causes the page to be moved in the active list. If, however, the second access does not occur within a given time interval after the first one, the page frame reclaiming algorithm may reset the `PG_referenced` flag.

As shown in Figure 17-4, the PFRA uses the `mark_page_accessed()`, `page_referenced()`, and `refill_inactive_zone()` functions to move the pages across the LRU lists. In the figure, the LRU list including the page is specified by the status of the `PG_active` flag.

Figure 17-4. Moving pages across the LRU lists



17.3.1.2. The `mark_page_accessed()` function

Whenever the kernel must mark a page as accessed, it invokes the `mark_page_accessed()` function. This happens every time the kernel determines that a page is being referenced by a User Mode process, a filesystem layer, or a device driver. For instance, `mark_page_accessed()` is invoked in the following cases:

- When loading on demand an anonymous page of a process (performed by the `do_anonymous_page()` function; see the section "Demand Paging" in Chapter 9).
- When loading on demand a page of a memory mapped file (performed by the `filemap_nopage()` function; see the section "Demand Paging for Memory Mapping" in Chapter 16).
- When loading on demand a page of an IPC shared memory region (performed by the `shmem_nopage()` function; see the section "IPC Shared Memory" in Chapter 19).
- When reading a page of data from a file (performed by the `do_generic_file_read()` function; see the section "Reading from a File" in Chapter 16).
- When swapping in a page (performed by the `do_swap_page()` function; see the section "Swapping in Pages" later in this chapter).
- When looking up a buffer page in the page cache (see the `__find_get_block()` function in the section "Searching Blocks in the Page Cache" in Chapter 15).

The `mark_page_accessed()` function executes the following code fragment:

```
if (!PageActive(page) && PageReferenced(page) && PageLRU(page)) {
    activate_page(page);
    ClearPageReferenced(page);
} else if (!PageReferenced(page))
```

```
SetPageReferenced(page);
```

As shown in [Figure 17-4](#), the function moves the page from the inactive list to the active list only if the `PG_referenced` flag is set before the invocation.

17.3.1.3. The `page_referenced()` function

The `page_referenced()` function, which is invoked once for every page scanned by the PFRA, returns 1 if either the `PG_referenced` flag or some of the `Accessed` bits in the Page Table entries was set; it returns 0 otherwise. This function first checks the `PG_referenced` flag of the page descriptor; if the flag is set, it clears it. Next, it makes use of the object-based reverse mapping mechanism to check and clear the `Accessed` bits in all User Mode Page Table entries that refer to the page frame. To do this, the function makes use of three ancillary functions; `page_referenced_anon()`, `page_referenced_file()`, and `page_referenced_one()`, which are analogous to the `try_to_unmap_xxx()` functions described in the section "[Reverse Mapping](#)" earlier in this chapter. The `page_referenced()` function also honors the swap token; see the section "[The Swap Token](#)" later in this chapter.

The `page_referenced()` function never moves a page from the active list to the inactive list; this job is done by `refill_inactive_zone()`. In practice, this function does a lot more than move pages from the active to the inactive list, so we are going to describe it in greater detail.

17.3.1.4. The `refill_inactive_zone()` function

As illustrated in [Figure 17-3](#), the `refill_inactive_zone()` function is invoked by `shrink_zone()`, which performs the reclaiming of pages in the page cache and in the User Mode address spaces (see the section "[Low On Memory Reclaiming](#)" later in this chapter). The function receives two parameters: a pointer `zone` to a memory zone descriptor, and a pointer `sc` to a `scan_control` structure. The latter data structure is widely used by the PFRA and contains information about the ongoing reclaiming operation; its fields are shown in [Table 17-2](#).

Table 17-2. The fields of the `scan_control` descriptor

Type	Field	Description
unsigned long	<code>nr_to_scan</code>	Target number of pages to be scanned in the active list.
unsigned long	<code>nr_scanned</code>	Number of inactive pages scanned in the current

			iteration.
			Number
			pages
unsigned long	nr_reclaimed		reclaimed in the current iteration.
			Number
			pages
unsigned long	nr_mapped		reference in the Us Mode address spaces.
			Target
int	nr_to_reclaim		number of pages to be reclaimed.
			Priority of the scanning ranging between 0 and 0.
unsigned int	priority		Lower priority implies scanning more pages.
			GFP mask passed from calling function.
unsigned int	gfp_mask		If set, writing a dirty page to disk is allowed (only for laptop mode).
int	may_writepage		

The role of `refill_inactive_zone()` is critical because moving a page from an active list to an inactive list means making the page eligible to fall prey, sooner or later, to the PFRA. If the function is too aggressive, it will move a lot of pages from the active list to the inactive list; as a consequence, the PFRA will reclaim a large number of page frames, and the system performance will be hit. On the other hand, if the function is too lazy, the inactive list will not be replenished with a large enough number of unused pages, and the PFRA will fail in reclaiming memory. Thus, the function implements an adaptive behavior: it starts by scanning, at every invocation, a small number of pages in the active list; however, if the PFRA is having trouble in reclaiming page frames, `refill_inactive_zone()` keeps increasing the number of active pages scanned at every invocation. This behavior is controlled by the value of the `priority` field in the

`scan_control` data structure (a lower value means a more urgent priority).

Another heuristic rule regulates the behavior of the `refill_inactive_zone( )` function. The LRU lists include two kinds of pages: those belonging to the User Mode address spaces, and those included in the page cache that do not belong to any User Mode process. As stated earlier, the PFRA should tend to shrink the page cache while leaving in RAM the pages owned by the User Mode processes. However, no fixed "golden rule" may yield good performance in every scenario, thus the `refill_inactive_zone( )` function relies on a swap tendency heuristic value: it determines whether the function will move all kinds of pages, or just the pages that do not belong to the User Mode address spaces.^[*] The swap tendency value is computed by the function as follows:

[*] The name "swap tendency" is a bit misleading, because the pages in User Mode address spaces can be swappable, syncable, or discardable. However, the swap tendency value certainly controls the amount of swapping performed by the PFRA, because almost all swappable pages belong to the User Mode address spaces.

$$\text{swap tendency} = \text{mapped ratio} / 2 + \text{distress} + \text{swappiness}$$

The mapped ratio value is the percentage of pages in all memory zones that belong to User Mode address spaces (`sc->nr_mapped`) with respect to the total number of allocatable page frames. A high value of `mapped_ratio` means that the dynamic memory is mostly used by User Mode processes, while a low value means that it is mostly used by the page cache.

The distress value is a measure of how effectively the PFRA is reclaiming page frames in this zone; it is based on the scanning priority of the zone in the previous run of the PFRA, which is stored in the `prev_priority` field of the zone descriptor. The distress value depends on the zone's previous priority as follows:

Zone prev. priority	12...7	6	5	4	3	2	1	0
Distress value	0	1	3	6	12	25	50	100

Finally, the swappiness value is a user-defined constant, which is usually set to 60. The system administrator may tune this value by writing in the `/proc/sys/vm/swappiness` file or by issuing the proper `sysctl( )` system call.

Pages will be reclaimed from the address spaces of processes only if the zone's swap tendency is greater than or equal to 100. Thus, if the system administrator sets swappiness to 0, then the PFRA never reclaims pages in the User Mode address spaces unless the zone's previous priority is zero (an unlikely event); if the administrator sets swappiness to 100, then the PFRA reclaims pages in the User Mode address spaces at every invocation.

Here is a succinct description of what the `refill_inactive_zone( )` function does:

1. Invokes `lru_add_drain( )` to move into the active and inactive lists any page still contained in the `pagevec` data structures.
2. Gets the `zone->lru_lock` spin lock.
3. Performs a first cycle scanning the pages in `zone->active_list`, starting from the tail of the list and moving backwards. Continues until the list is empty or until `sc->nr_to_scan` pages have been scanned. For each page scanned in this cycle, the function increases by one its reference counter, removes the page descriptor from `zone->active_list`, and puts it in a temporary `l_hold` local list. However, if the reference counter of the page frame was zero, it puts back the page in the active list. In fact, page frames having a reference counter equal to zero should belong to the zone's Buddy

system; however, to free a page frame, first its usage counter is decreased and then the page frame is removed from the LRU lists and inserted in the buddy system's list. Therefore, there is a small time window in which the PFRA may see a free page in an LRU list.

4. Adds to `zone->pages_scanned` the number of active pages that have been scanned.
5. Subtracts from `zone->nr_active` the number of pages that have been moved into the `l_hold` local list.
6. Releases the `zone->lru_lock` spin lock.
7. Computes the swap tendency value (see above).
8. Performs a second cycle on the pages in the `l_hold` local list. The objective of this cycle is to split the pages of `l_hold` into two local sublists called `l_active` and `l_inactive`. A page belonging to the User Mode address space of some process that is, a page whose `page->_mapcount` is nonnegative is added to `l_active` if the swap tendency value is smaller than 100, or if the page is anonymous but no swap area is active, or finally if the `page_referenced()` function applied to the page returns a positive value, which means that the page has been recently accessed. In all other cases, the page is added to the `l_inactive` list.^[*]

[*] Notice that a page that does not belong to any User Mode process address space is moved into the inactive list; however, since its `PG_referenced` flag is not cleared, the first access to the page causes the `mark_page_accessed()` function to move the page back into the active list (see Figure 17-4).

9. Gets the `zone->lru_lock` spin lock.
10. Performs a third cycle on the pages in the `l_inactive` local list to move them in the `zone->inactive_list` list and updates the `zone->nr_inactive` field. In doing so, it decreases the usage counters of the moved page frames to undo the increments done in step 3.
11. Performs a fourth and last cycle on the pages in the `l_active` local list to move them into the `zone->active_list` list and updates the `zone->nr_active` field. In doing so, it decreases the usage counters of the moved page frames to undo the increments done in step 3.
12. Releases the `zone->lru_lock` spin lock and returns.

It should be noted that `refill_inactive_zone()` checks the `PG_referenced` flag only for pages that belong to the User Mode address spaces (see step 8); in the opposite case, the pages are in the tail of the active list hence they were accessed some time ago and it is unlikely that they will be accessed in the near future. On the other hand, the function does not evict a page from the active list if it is owned by some User Mode process and has been recently used.

17.3.2. Low On Memory Reclaiming

Low on memory reclaiming is activated when a memory allocation fails. As shown in Figure 17-3, the kernel invokes `free_more_memory()` while allocating a VFS buffer or a buffer head, and it invokes `try_to_free_pages()` while allocating one or more page frames from the buddy system.

17.3.2.1. The `free_more_memory()` function

The `free_more_memory()` function performs the following actions:

1. Invokes `wakeup_bdflush()` to wake a `pdflush` kernel thread and trigger write operations for 1024 dirty pages in the page cache (see the section "The `pdflush` Kernel Threads" in Chapter 15). Writing dirty pages to disk may eventually make freeable the page frames containing buffers, buffer heads, and other VFS data structures.
2. Invokes the service routine of the `sched_yield()` system call to give the `pdflush` kernel thread a chance to run.

3. Starts a loop over all memory nodes in the system (see the section "[Non-Uniform Memory Access \(NUMA\)](#)" in [Chapter 8](#)). For each node, invokes the `try_to_free_pages( )` function passing to it a list of the "low" memory zones (in the 80 x 86 architecture, `ZONE_DMA` and `ZONE_NORMAL`; see the section "[Memory Zones](#)" in [Chapter 8](#)).

17.3.2.2. The `try_to_free_pages( )` function

The `try_to_free_pages( )` function receives three parameters:

`zones`

A list of memory zones in which pages should be reclaimed (see the section "[Memory Zones](#)" in [Chapter 8](#))

`gfp_mask`

The set of allocation flags that were used by the failed memory allocation (see the section "[The Zoned Page Frame Allocator](#)" in [Chapter 8](#))

`order`

Not used

The goal of the function is to free at least 32 page frames by repeatedly invoking the `shrink_caches( )` and `shrink_slab( )` functions, each time with a higher priority than the previous invocation. The ancillary functions get the priority level as well as other parameters of the ongoing scan operation in a descriptor of type `scan_control` (see [Table 17-2](#) earlier in this chapter). The lowest, initial priority level is 12, while the highest, final priority level is 0. If `try_to_free_pages( )` does not succeed in reclaiming at least 32 page frames in one of the 13 repeated invocations of `shrink_caches( )` and `shrink_slab( )`, the PFRA is in serious trouble, and it has just one last resort: killing a process to free all its page frames. This operation is performed by the `out_of_memory( )` function (see the section "[The Out of Memory Killer](#)" later in this chapter).

The function performs the following main steps:

1. Allocates and initializes a `scan_control` descriptor. In particular, stores the `gfp_mask` allocation mask in the `gfp_mask` field.
2. For each zone in the `zones` lists, it sets the `temp_priority` field of the zone descriptor to the initial priority (12). Moreover, it computes the total number of pages contained in the LRU lists of the zones.
3. Performs a loop of at most 13 iterations, from priority 12 down to 0; in each iteration performs the following substeps:
 - a. Updates some field of the `scan_control` descriptor. In particular, it stores in the `nr_mapped` field the total number of pages owned by User Mode processes, and in the `priority` field the current priority of this iteration. Also, it sets to zero the `nr_scanned` and `nr_reclaimed` fields.
 - b. Invokes `shrink_caches( )` passing as arguments the `zones` list and the address of the `scan_control` descriptor. This function scans the inactive pages of the zones (see below).
 - c. Invokes `shrink_slab( )` to reclaim pages from the shrinkable kernel caches (see the section "[Reclaiming Pages of Shrinkable Disk Caches](#)" later in this chapter).

- d. If the `current->reclaim_state` field is not NULL, it adds to the `nr_reclaimed` field of the `scan_control` descriptor the number of pages reclaimed from the slab allocator caches; this number is stored in a small data structure pointed to by the process descriptor field. The `_ _alloc_pages( )` function sets up the `current->reclaim_state` field before invoking the `TRy_to_free_pages( )` function, and clears the field right after its termination. (Oddly, the `free_more_memory( )` function does not set this field.)
- e. If the target has been reached (the `nr_reclaimed` field of the `scan_control` descriptor is greater than or equal to 32), it breaks the loop and jumps to step 4.
- f. The target has not yet been reached. If at least 49 pages have been scanned so far, the function invokes `wakeup_bdflush( )` to activate a `pdflush` kernel thread and write some dirty pages in the page cache to disk (see the section "[Looking for Dirty Pages To Be Flushed](#)" in [Chapter 15](#)).
- g. If the function has already performed four iterations without reaching the target, it invokes `blk_congestion_wait( )` to suspend the current process until any `WRITE` request queue becomes uncongested or until a 100 ms time-out elapses (see the section "[Request Descriptors](#)" in [Chapter 14](#)).
- 4. Sets the `prev_priority` field of each zone descriptor to the priority level used in the last invocation of `shrink_caches( )`; it is stored in the `temp_priority` field of the zone descriptor.
- 5. Returns 1 if the reclaiming was successful, 0 otherwise.

17.3.2.3. The `shrink_caches( )` function

The `shrink_caches( )` function is invoked by `TRy_to_free_pages( )`. It acts on two parameters: the zones list of memory zones, and the address `sc` of a `scan_control` descriptor.

The purpose of this function is simply to invoke the `shrink_zone( )` function on each zone in the zones list. However, before invoking `shrink_zone( )` on a given zone, `shrink_caches( )` updates the `temp_priority` field of the zone's descriptor by using the value stored in the `sc->priority` field; this is the current priority level of the scanning operation. Moreover, if the priority value of the previous invocation of the PFRA is higher than the current priority value that is, page frame reclaiming in this zone is now harder to do `shrink_caches( )` copies the current priority level into the `prev_priority` field of the zone descriptor. Finally, `shrink_caches( )` does not invoke `shrink_zone( )` on a given zone if the `all_unreclaimable` flag in the zone descriptor is set and the current priority level is less than 12 that is, `shrink_caches( )` is not being invoked in the very first iteration of `try_to_free_pages( )`. The PFRA sets the `all_unreclaimable` flag when it decides that a zone is so full of unreclaimable pages that scanning the zone's pages is just a waste of time.

17.3.2.4. The `shrink_zone( )` function

The `shrink_zone( )` function acts on two parameters: `zone`, a pointer to a `struct_zone` descriptor, and `sc`, a pointer to a `scan_control` descriptor. The goal of this function is to reclaim 32 pages from the zone's inactive list; the function tries to reach this goal by invoking repeatedly an auxiliary function called `shrink_cache( )`, each time on larger portion of the zone's inactive list. Moreover, `shrink_zone( )` replenishes the zone's inactive list by repeatedly invoking the `refill_inactive_zone( )` function described in the earlier section "[The Least Recently Used \(LRU\) Lists](#)."

The `nr_scan_active` and `nr_scan_inactive` fields of the zone descriptor play a special role here. To be efficient, the function works on batches of 32 pages. Thus, if the function is running at a low privilege level (high value of `sc->priority`) and one of the LRU lists does not contain enough pages, the function skips

the scanning on that list. However, the number of active or inactive pages thus skipped is recorded in `nr_scan_active` or `nr_scan_inactive`, so that the skipped pages will be considered in the next invocation of the function.

Specifically, the `shrink_zone( )` function performs the following steps:

1. Increases the `zone->nr_scan_active` by a fraction of the total number of elements in the active list (`zone->nr_active`). The actual increment is determined by the current priority level and ranges from `zone->nr_active/212` to `zone->nr_active/20` (i.e., the whole number of active pages in the zone).
2. Increases the `zone->nr_scan_inactive` by a fraction of the total number of elements in the active list (`zone->nr_inactive`). The actual increment is determined by the current priority level and ranges from `zone->nr_inactive/212` to `zone->nr_inactive`.
3. If the `zone->nr_scan_active` field is greater than or equal to 32, the function copies its value in the `nr_active` local variable and sets the field to zero; otherwise, it sets `nr_active` to zero.
4. If the `zone->nr_scan_inactive` field is greater than or equal to 32, the function copies its value in the `nr_inactive` local variable and sets the field to zero; otherwise, it sets `nr_inactive` to zero.
5. Sets the `sc->nr_to_reclaim` field of the `scan_control` descriptor to 32.
6. If both `nr_active` and `nr_inactive` are 0, there is nothing to be done: the function terminates. This is an unlikely situation where User Mode processes have no page frames allocated to them.
7. If `nr_active` is positive, it replenishes the zone's inactive list:

```
sc->nr_to_scan = min(nr_active, 32);
nr_active -= sc->nr_to_scan;
refill_inactive_zone(zone, sc);
```

8. If `nr_inactive` is positive, it tries to reclaim at most 32 pages from the inactive list:

```
sc->nr_to_scan = min(nr_inactive, 32);
nr_inactive -= sc->nr_to_scan;
shrink_cache(zone, sc);
```

9. If `shrink_zone( )` succeeds in reclaiming 32 pages (`sc->nr_to_reclaim` is now zero or negative), it terminates. Otherwise, it jumps back to step 6.

17.3.2.5. The `shrink_cache( )` function

The `shrink_cache( )` function is yet another auxiliary function whose main purpose is to extract from the zone's inactive list a group of pages, put them in a temporary list, and invoke the `shrink_list( )` function to effectively perform page frame reclaiming on every page in that list. The `shrink_cache( )` function acts on the same parameters as `shrink_zones( )`, namely `zone` and `sc`, and performs the following main steps:

1. Invokes `lru_add_drain( )` to move into the active and inactive lists any page still contained in the `pagevec` data structures (see the section "[The Least Recently Used \(LRU\) Lists](#)" earlier in this chapter).
2. Gets the `zone->lru_lock` spin lock.
3. Considers at most 32 pages in the inactive list; for each page, the function increases its usage counter, checks whether the page is not being freed to the buddy system (see the discussion at step 3 of `refill_inactive_zone( )`), and moves the page from the zone's inactive list to a local list.

4. Decreases the counter `zone->nr_inactive` by the number of pages removed from the inactive list.
5. Increases the counter `zone->pages_scanned` by the number of pages effectively examined in the inactive list.
6. Releases the `zone->lru_lock` spin lock.
7. Invokes the `shrink_list( )` function passing to it the (local list of) pages collected in step 3 above. This function is discussed below (as you were no doubt expecting).
8. Decreases the `sc->nr_to_reclaim` field by the number of pages actually reclaimed by `shrink_list( )`.
9. Gets again the `zone->lru_lock` spin lock.
10. Puts back in the inactive or active list all pages of the local list that `shrink_list( )` did not succeed in freeing. Notice that `shrink_list( )` might mark a page as active by setting its `PG_active` flag. This operation is performed in a batch of pages using a `pagevec` data structure (see the section "[The Least Recently Used \(LRU\) Lists](#)" earlier in this chapter).
11. If the function scanned at least `sc->nr_to_scan` pages, and if it didn't succeed in reclaiming the target number of pages (i.e., `sc->nr_to_reclaim` is still positive), it jumps back to step 3.
12. Releases the `zone->lru_lock` spin lock and terminates.

17.3.2.6. The `shrink_list( )` function

We have now reached the heart of page frame reclaiming. While the purpose of the functions illustrated so far, from `try_to_free_pages( )` to `shrink_cache( )`, was to select the proper set of pages candidates for reclaiming, the `shrink_list( )` function effectively tries to reclaim the pages passed as a parameter in the `page_list` list. The second parameter, namely `sc`, is the usual pointer to a `scan_control` descriptor. When `shrink_list( )` returns, `page_list` contains the pages that couldn't be freed.

The function performs the following actions:

1. If the `need_resched` field of the current process is set, it invokes `schedule( )`.
2. Starts a cycle on every page descriptor included in the `page_list` list. For each list item, it removes the page descriptor from the list and tries to reclaim the page frame; if for some reason the page frame could not be freed, it inserts the page descriptor in a local list.
3. Now the `page_list` list is empty: the function moves back the page descriptors from the local list to the `page_list` list.
4. Increases the `sc->nr_reclaimed` field by the number of page frames reclaimed in step 2, and returns that number.

Of course, what is really interesting in `shrink_list( )` is the code that tries to reclaim a page frame. The flow diagram of this code is shown in [Figure 17-5](#).

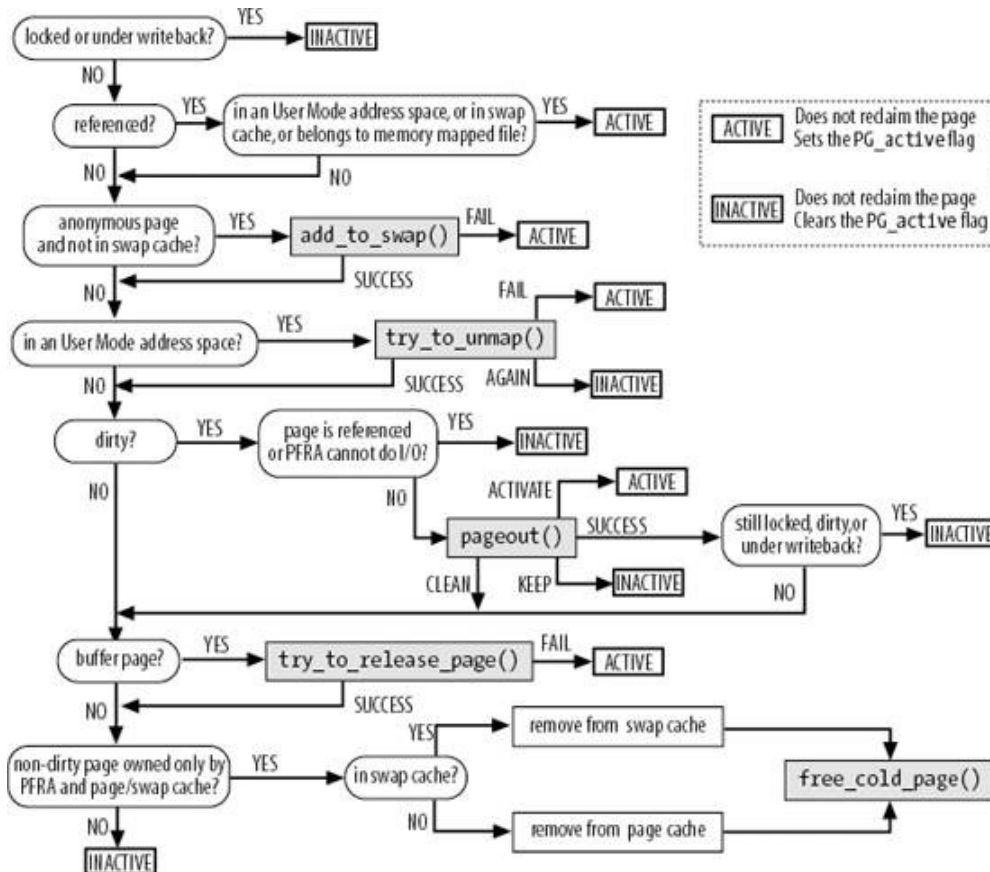
There are only three possible outcomes for each page frame handled by `shrink_list( )`:

- The page is released to the zone's buddy system by invoking the `free_cold_page( )` function (see the section "[The Per-CPU Page Frame Cache](#)" in [Chapter 8](#)); hence, the page is effectively reclaimed.
- The page is not reclaimed, thus it will be reinserted in the `page_list` list; however, `shrink_list( )` assumes that it will be possible to reclaim the page in the near future. Thus, the function leaves the `PG_active` flag in the page descriptor cleared, so that the page will be put back in the inactive list of the memory zone (see step 9 in the descriptor of `shrink_cache( )` above). This event corresponds to the small boxes labeled as "INACTIVE" in [Figure 17-5](#).
- The page is not reclaimed, thus it will be reinserted in the `page_list` list; however, either the page is in active use, or `shrink_list( )` assumes that it will be impossible to reclaim the page in the

foreseeable future. Thus, the function sets the `PG_active` flag in the page descriptor, so that the page will be put in the active list of the memory zone. This event corresponds to the small boxes labeled as "ACTIVE" in [Figure 17-5](#).

The `shrink_list( )` function never tries to reclaim a page that is locked (`PG_locked` flag set) or under writeback (`PG_writeback` flag set). In order to test whether the page was recently referenced, `shrink_list( )` invokes `page_referenced( )`, which was described in the section "[The Least Recently Used \(LRU\) Lists](#)" earlier in this chapter.

Figure 17-5. The page reclaiming logic of the shrink_list() function



To reclaim an anonymous page, the page must be added to the swap cache, and a new slot in a swap area must be reserved for it; see the section "[Swapping](#)" later in this chapter for details.

If the page is in the User Mode address space of some process (the `_mapcount` field in the page descriptor is greater than or equal to zero), `shrink_list( )` invokes the `try_to_unmap( )` function to locate all User Mode Page Table entries that refer to the page frame (see the section "[Reverse Mapping](#)" earlier in this chapter). Of course, reclaiming may proceed only if this function returns `SWAP_SUCCESS`.

If the page is dirty, it cannot be reclaimed unless it is written to disk. To do this, `shrink_list()` relies on the `pageout()` function, which is described next. The reclaiming of the page frame may proceed only if either `pageout()` does not have to issue a write operation, or if the write operation finishes soon.

If the page contains VFS buffers, `shrink_list()` invokes `TRY_to_release_page()` to release the associated buffer heads (see the section "[Releasing Block Device Buffer Pages](#)" in [Chapter 15](#)).

Finally, if everything went smoothly, `shrink_list()` checks the reference counter of the page: if it is equal to two, the page has just two owners: the page cache (or the swap cache, in case of anonymous pages), and the PFRA itself (the reference counter was increased in step 3 of `shrink_cache()`; see earlier). In this case, the page can be reclaimed, provided it is still not dirty. To do this, first the page is removed from the page cache or the swap cache, according to the value of the `PG_swapcache` flag of the page descriptor; then, the `free_cold_page()` function is executed.

17.3.2.7. The `pageout()` function

The `pageout()` function is invoked by `shrink_list()` when a dirty page must be written to disk. Essentially, the function performs the following operations:

1. Checks that the page is included in the page cache or in the swap cache (see the section "[The Swap Cache](#)" later in this chapter). Moreover, checks that the page is owned only by the page cache or the swap cache and the PFRA. Returns `PAGE_KEEP` if a check has failed (it does not make sense to write the page to disk if it is not reclaimable by `shrink_list()`).
2. Checks that the `writepage` method of the `address_space` object is defined; returns `PAGE_ACTIVATE` otherwise.
3. Checks that the current process can issue write requests to the request queue of the block device associated with the `address_space` object. Essentially, the `kswapd` and `pdflush` kernel threads may always issue the write request; normal processes can issue the write request only if the request queue is not congested, unless the `current->backing_dev_info` field points to the `backing_dev_info` data structure of the block device (see step 3 of the description of the `generic_file_aio_write_nolock()` function in the section "[Writing to a File](#)" in [Chapter 16](#)).
4. Checks that the page is still dirty; if not, returns `PAGE_CLEAN`.
5. Sets up a `writeback_control` descriptor and invokes the `writepage` method of the `address_space` object to start a write back operation (see the section "[Writing Dirty Pages to Disk](#)" in [Chapter 16](#)).
6. If the `writepage` method returned an error code, the function returns `PAGE_ACTIVATE`.
7. Returns `PAGE_SUCCESS`.

17.3.3. Reclaiming Pages of Shrinkable Disk Caches

We know from the previous chapters that the kernel uses other disk caches beside the page cache, for instance the dentry cache and the inode cache (see the section "[The dentry Cache](#)" in [Chapter 12](#)). When the PFRA tries to reclaim page frames, it should also check whether some of these disk caches can be shrunk.

Every disk cache that is considered by the PFRA must have a shrinker function registered at initialization time. The shrinker function expects two parameters: the target number of page frames to be reclaimed, and a set of GFP allocation flags; the function does what is required to reclaim the pages from the disk cache, then it returns the number of reclaimable pages remaining in the cache.

The `set_shrinker()` function registers a shrinker function with the PFRA. This function allocates a descriptor of type `shrinker`, stores the address of the shrinker function in the descriptor, and then inserts the descriptor in a global list rooted at the `shrinker_list` global variable. The `set_shrinker()` function also initializes the `seeks` field of the `shrinker` descriptor: informally, it is a parameter that indicates how much it costs to re-create one item of the cache once it is removed.

In Linux 2.6.11 there are few disk caches registered with the PFRA: besides the dentry cache and the inode cache, only the disk quota layer, the filesystem meta information block cache (mainly used for filesystems'

extended attributes), and the XFS journaling filesystem register shrinker functions .

The PFRA's function that reclaims pages from the shrinkable disk caches is called `shrink_slab( )` (the name is a bit misleading, because the function has little to do with the slab allocator caches). This function is invoked by `Try_to_free_pages( )`, as explained in the earlier section "[Low On Memory Reclaiming](#)," and by `balance_pgdat( )`, which is described in the later section "[Periodic Reclaiming](#)."

The `shrink_slab( )` function tries to balance the cost of reclaiming from the shrinkable disk cache with the cost of reclaiming from the LRU lists (performed by `shrink_list( )`). Essentially, the function walks the list in the `shrinker` descriptors to invoke the shrinker functions and get the total number of reclaimable pages in the disk caches. Then, the function scans again the list of the `shrinker` descriptor; for each shrinkable disk cache, the function heuristically computes a target number of page frames to be reclaimed based on the number of reclaimable pages in the disk caches, on the relative cost of re-creating a page in the disk cache, and on the number of pages in the LRU lists and invokes the shrinker function to try to reclaim batches of at least 128 pages.

For lack of space, we'll limit ourselves to describe briefly the shrinker functions of the dentry cache and of the inode cache.

17.3.3.1. Reclaiming page frames from the dentry cache

The `shrink_dcache_memory( )` function is the shrinker function for the dentry cache; it searches the cache for unused dentry objects that is, objects not referenced by any process, see the section "[dentry Objects](#)" in [Chapter 12](#) and releases them.

Because the dentry cache objects are allocated through the slab allocator, the `shrink_dcache_memory( )` function may lead some slabs to become free, causing some page frames to be consequently reclaimed by `cache_reap( )` (see the section "[Periodic Reclaiming](#)" later in this chapter). Moreover, the dentry cache acts as a controller of the inode cache. Therefore, when a dentry object is released, the pages storing the corresponding inode may become unused, and thus eventually released.

The `shrink_dcache_memory( )` function receives as its parameters the number of page frames to reclaim and a GFP mask. It starts by checking whether the `__GFP_FS` bit in the GFP mask is clear; if so, the function returns `-1`, because releasing a dentry may trigger an operation on a disk-based filesystem. Page frame reclaiming is effectively done by invoking `prune_dcache( )`. This function scans the list of unused dentries whose head is stored in the `dentry_unused` variable until it reaches the requested number of freed objects or until the whole list is scanned. On each object that wasn't recently referenced, the function:

1. Removes the dentry object from the dentry hash table, from the list of dentry objects in its parent directory, and from the list of dentry objects of the owner inode.
2. Decreases the usage counter of the dentry's inode by invoking the `d_iput` dentry method, if defined, or the `iput( )` function.
3. Invokes the `d_release` method of the dentry object, if defined.
4. Invokes the `call_rcu( )` function to register a callback function that will remove the dentry object (see the section "[Read-Copy Update \(RCU\)](#)" in [Chapter 5](#)). The callback function, in turn, will invoke `kmem_cache_free( )` to release the object to the slab allocator (see the section "[Freeing a Slab Object](#)" in [Chapter 8](#)).
5. Decreases the usage counter of the parent directory.

Finally, `shrink_dcache_memory( )` returns a value based on the number of unused dentries still contained in the dentry cache. More precisely, the returned value is the number of unused dentries multiplied by 100 and divided by the content of the `sysctl_vfs_cache_pressure` global variable. By default,

this variable is equal to 100, thus the returned value is essentially the number of unused dentries. However, the system administrator may modify the variable by writing in the `/proc/sys/vm/vfs_cache_pressure` or by issuing a suitable `sysctl( )` system call. Setting this variable to a value smaller than 100 causes `shrink_slab( )` to reclaim fewer pages from the dentry cache (and the inode cache; see the next section) with respect to the pages reclaimed from the LRU lists; conversely, setting the variable to a value greater than 100 causes `shrink_slab( )` to reclaim more pages from the dentry and inode caches with respect to the pages reclaimed from the LRU lists.

17.3.3.2. Reclaiming page frames from the inode cache

The `shrink_ichache_memory( )` function is invoked to remove unused inode objects from the inode cache; here, "unused" means that the inode no longer has a controlling dentry object. The function is similar to the `shrink_dcache_memory( )` described previously. It checks the `__GFP_FS` bit in the `gfp_mask` parameter, then it invokes the `prune_ichache( )` function, and finally it returns a value based both on the number of unused inodes still included in the inode cache and the value of the `sysctl_vfs_cache_pressure` variable, as previously.

The `prune_ichache( )` function, in turn, scans the `inode_unused` list (see the section "[Inode Objects](#)" in [Chapter 12](#)); to free an inode, the function releases any private buffer associated with the inode, it invalidates the clean page frames in the page cache that refer to the inode and are not longer in use, and then it makes use of the `clear_inode( )` and `destroy_inode( )` functions to destroy the inode object.

17.3.4. Periodic Reclaiming

The PFRA performs periodic reclaiming by using two different mechanisms: the `kswapd` kernel threads, which invoke `shrink_zone( )` and `shrink_slab( )` to reclaim pages from the LRU lists, and the `cache_reap` function, which is invoked periodically to reclaim unused slabs from the slab allocator.

17.3.4.1. The `kswapd` kernel threads

The `kswapd` kernel threads are another kernel mechanism that activates page frame reclaiming. Why is it necessary? Is it not sufficient to invoke `Try_to_free_pages( )` when free memory becomes really scarce and another memory allocation request is issued?

Unfortunately, this is not the case. Some memory allocation requests are performed by interrupt and exception handlers, which cannot block the current process waiting for a page frame to be freed; moreover, some memory allocation requests are done by kernel control paths that have already acquired exclusive access to critical resources and that, therefore, cannot activate I/O data transfers. In the infrequent case in which all memory allocation requests are done by such sorts of kernel control paths, the kernel is never able to free memory.

The `kswapd` kernel threads also have a beneficial effect on system performance by keeping memory free in what would otherwise be idle time for the machine; processes can thus get their pages much faster.

There is a different `kswapd` kernel thread for each memory node (see the section "[Non-Uniform Memory Access \(NUMA\)](#)" in [Chapter 8](#)). Each such thread is usually sleeping in the wait queue headed at the `kswapd_wait` field of the node descriptor. However, if `__alloc_pages( )` discovers that all memory zones suitable for a memory allocation have a number of free page frames below a "warning"

threshold essentially, a value based on the `pages_low` and `protection` fields of the memory zone descriptor then the function wakes up the `kswapd` kernel threads of the corresponding memory nodes (see the section "[The Zone Allocator](#)" in [Chapter 8](#).) Essentially, the kernel starts to reclaim some page frames in order to avoid much more dramatic "low on memory" conditions.

As explained in the section "[The Pool of Reserved Page Frames](#)" in [Chapter 8](#), every zone descriptor also includes a `pages_min` field a threshold that specifies the minimum number of free page frames that should always be preserved and a `pages_high` field a threshold that specifies the "safe" number of free page frames above which page frame reclaiming should be stopped.

The `kswapd` kernel thread executes the `kswapd( )` function. It initializes the kernel thread by binding the kernel thread to the CPUs that may access the memory node, by storing in the `current->reclaim_state` field of the process descriptor the address of a `reclaim_state` descriptor (see step 3d in the description of `TRY_to_free_pages( )` earlier in this chapter), and by setting the `PF_MEMALLOC` and `PF_KSWAP` flags in the `current->flags` field these flags indicate that the process is reclaiming memory and that it is allowed to use all the free memory available when doing its job. Every time the `kswapd` kernel thread is awakened, the `kswapd( )` function performs essentially the following steps:

1. Invokes `finish_wait( )` to remove the kernel thread from the node's `kswapd_wait` wait queue (see the section "[How Processes Are Organized](#)" in [Chapter 3](#)).
2. Invokes `balance_pgdat( )` to perform the memory reclaiming on the `kswapd`'s memory node (see below).
3. Invokes `prepare_to_wait( )` to set the process in the `TASK_INTERRUPTIBLE` state and to put it to sleep in the node's `kswapd_wait` wait queue.
4. Invokes `schedule( )` to yield the CPU to some other runnable process.

The `balance_pgdat( )` function performs, in turn, the following basic steps:

1. Sets up a `scan_control` descriptor (see [Table 17-2](#) earlier in this chapter).
2. Sets the `temp_priority` field of each zone descriptor in the memory node to 12 (lowest priority).
3. Performs a loop of at most 13 iterations, from priority 12 down to 0; in each iteration performs the following substeps:
 - a. Scans the memory zones to find the highest zone (from `ZONE_DMA` to `ZONE_HIGHMEM`) having an insufficient number of free page frames. Each test is done by executing the `zone_watermark_ok( )` function described in the section "[The Zone Allocator](#)" in [Chapter 8](#). If all zones have a large number of free page frames, it jumps to step 4.
 - b. Scans again the memory zones proceeding from `ZONE_DMA` to the zone found in step 3a. For each zone, it updates, if necessary, the `prev_priority` field of the zone descriptor with the current priority level, and invokes successively `shrink_zone( )` to reclaim pages from the zone (see the earlier section "[Low On Memory Reclaiming](#)"). Next, it invokes `shrink_slab( )` to reclaim pages from the shrinkable disk caches (see the earlier section "[Reclaiming Pages of Shrinkable Disk Caches](#)").
 - c. If at least 32 pages have been reclaimed, it breaks the loop and jumps to step 4.
4. Updates the `prev_priority` field of each zone descriptor with the value stored in the corresponding `temp_priority` field.
5. If some "low on memory" zone still exists, it invokes `schedule( )` if the `need_resched` field of the process is set; when in execution again, it jumps back to step 1.
6. Returns the number of pages reclaimed.

17.3.4.2. The `cache_reap()` function

The PFRA must also reclaim the pages owned by the slab allocator caches (see the section "[Memory Area Management](#)" in [Chapter 8](#)). To do this, it relies on the `cache_reap()` function, which is periodically scheduled approximately once every two seconds in the predefined events work queue (see the section "[Work Queues](#)" in [Chapter 4](#)). The address of the `cache_reap()` function is stored in the `func` field of the `reap_work` per-CPU variable of type `work_struct`.

The `cache_reap()` function essentially performs the following steps:

1. Tries to acquire the `cache_chain_sem` semaphore, which protects the list of slab cache descriptors; if the semaphore is already taken, it invokes `schedule_delayed_work()` to schedule the next invocation of the function, and terminates.
2. Otherwise, scans the `kmem_cache_t` descriptors collected in the `cache_chain` list. For each cache descriptor found, the function performs the following steps:
 - a. If the `SLAB_NO_REAP` flag in the cache descriptor is set, page frame reclaiming has been disabled, hence it continues with the next cache in the list.
 - b. Drains the slab local cache (see the section "[Local Caches of Free Slab Objects](#)" in [Chapter 8](#)); this operation could cause new slabs to become free.
 - c. Each cache has a "reap time" stored in the `next_reap` field of the `kmem_list3` structure inside the cache descriptor (see the section "[Cache Descriptor](#)" in [Chapter 8](#)); if `jiffies` is still smaller than `next_reap`, it continues with the next cache in the list.
 - d. Sets the next "reap time" in the `next_reap` field to a value four seconds from the current time.
 - e. In multiprocessor systems, the function drains the slab shared cache (see the section "[Local Caches of Free Slab Objects](#)" in [Chapter 8](#)); this operation could cause new slabs to become free.
 - f. If a new slab has been recently added to the cache that is, if the `free_touched` flag of the `kmem_list3` structure inside the cache descriptor is set it skips this cache and continues with the next cache in the list.
 - g. Computes according to a heuristic formula the number of slabs to be freed. Basically, this number depends on the upper limit of free objects in the cache and on the number of objects packed into a single slab.
 - h. Repeatedly invokes `slab_destroy()` on the slabs included in the list of free slabs of the cache, until the list is empty or the target number of free slab has been reached.
 - i. Invokes `cond_resched()` to check the `TIF_NEED_RESCHED` flag of the current process and to invoke `schedule()`, if the flag is set.
3. Releases the `cache_chain_sem` semaphore.
4. Invokes `schedule_delayed_work()` to schedule the next invocation of the function, and terminates.

17.3.5. The Out of Memory Killer

Despite the PFRA effort to keep a reserve of free page frames, it is possible for the pressure on the virtual memory subsystem to become so high that all available memory becomes exhausted. This situation could quickly induce a freeze of every activity in the system: the kernel keeps trying to free memory in order to satisfy some urgent request, but it does not succeed because the swap areas are full and all disk caches have already been shrunk. As a consequence, no process can proceed with its execution, thus no process will eventually free up the page frames that it owns.

To cope with this dramatic situation, the PFRA makes use of a so-called out of memory (OOM) killer, which selects a process in the system and abruptly kills it to free its page frames. The OOM killer is like a surgeon that amputates the limb of a man to save his life: losing a limb is not a nice thing, but sometimes there is nothing better to do.

The `out_of_memory()` function is invoked by `_alloc_pages()` when the free memory is very low and the PFRA has not succeeded in reclaiming any page frames (see the section "[The Zone Allocator](#)" in [Chapter 8](#)). The function invokes `select_bad_process()` to select a victim among the existing processes, then invokes `oom_kill_process()` to perform the sacrifice.

Of course, `select_bad_process()` does not simply pick a process at random. The selected process should satisfy several requisites:

- The victim should own a large number of page frames, so that the amount of memory that can be freed is significant. (As a countermeasure against the "fork-bomb" processes, the function considers the amount of memory eaten by all children owned by the parent, too.)
- Killing the victim should lose a small amount of work it is not a good idea to kill a batch process that has been working for hours or days.
- The victim should be a low static priority process the users tend to assign lower priorities to less important processes.
- The victim should not be a process with root privileges they usually perform important tasks.
- The victim should not directly access hardware devices (such as the X Window server), because the hardware could be left in an unpredictable state.
- The victim cannot be swapper (process 0), init (process 1), or any other kernel thread.

The `select_bad_process()` function scans every process in the system, uses an empirical formula to compute from the above rules a value that denotes how good selecting that process is, and returns the process descriptor address of the "best" candidate for eviction. Then, the `out_of_memory()` function invokes `oom_kill_process()` to send a deadly signal usually `SIGKILL`; see [Chapter 11](#) either to a child of that process or, if this is not possible, to the process itself. The `oom_kill_process()` function also kills all clones that share the same memory descriptor with the selected victim.

17.3.6. The Swap Token

As you might have realized while reading this chapter, the Linux VM subsystem and particularly the PFRA is so complex a piece of code that is quite hard to predict its behavior with an arbitrary workload. There are cases, moreover, in which the VM subsystem exhibits pathological behaviors. An example is the so-called swap thrashing phenomenon: essentially, when the system is short of free memory, the PFRA tries hard to free memory by writing pages to disk and stealing the underlying page frames from some processes; at the same time, however, these processes want to proceed with their executions, hence they try hard to access their pages. As a consequence, the kernel assigns to the processes the page frames just freed by the PFRA and reads their contents from disk. The net result is that pages are continuously written to and read back from the disk; most of the time is spent accessing the disk, hence no process makes substantial progress towards its termination.

To mitigate the likelihood of swap thrashing, a technique proposed by Jiang and Zhang in 2004 has been implemented in the kernel version 2.6.9: essentially, a so-called swap token is assigned to a single process in the system; the token exempts the process from the page frame reclaiming, so the process can make substantial progress and, hopefully, terminate even when memory is scarce.

The swap token is implemented as a `swap_token_mm` memory descriptor pointer. When a process owns the

swap token, `swap_token_mm` is set to the address of the process's memory descriptor.

Immunity from page frame reclaiming is granted in an elegant and simple way. As we have seen in the section "[The Least Recently Used \(LRU\) Lists](#)," a page is moved from the active to the inactive list only if it was not recently referenced. The check is done by the `page_referenced( )` function, which honors the swap token and returns 1 (referenced) if the page belongs to a memory region of the process that owns the swap token. Actually, in a couple of cases the swap token is not considered: when the PFRA is executing on behalf of the process that owns the swap token, and when the PFRA has reached the hardest priority level in page frame reclaiming (level 0).

The `grab_swap_token( )` function determines whether the swap token should be assigned to the current process. It is invoked on each major page fault, namely on just two occasions:

- When the `filemap_nopage( )` function discovers that the required page is not in the page cache (see the section "[Demand Paging for Memory Mapping](#)" in [Chapter 16](#)).
- When the `do_swap_page( )` function has read a new page from a swap area (see the section "[Swapping in Pages](#)" later in this chapter).

The `grab_swap_token( )` function makes some checks before assigning the token. In particular, the token is granted if all of the following conditions hold:

- At least two seconds have elapsed since the last invocation of `grab_swap_token( )`.
- The current token-holding process has not raised a major page fault since the last execution of `grab_swap_token( )`, or has been holding the token since at least `swap_token_default_timeout` ticks.
- The swap token has not been recently assigned to the current process.

The token holding time should ideally be rather long, even in the order of minutes, because the goal is to allow a process to finish its execution. In Linux 2.6.11 the token holding time is set by default to a very low value, namely one tick. However, the system administrator can tune the value of the `swap_token_default_timeout` variable by writing in the `/proc/sys/vm/swap_token_default_timeout` file or by issuing a proper `sysctl( )` system call.

When a process is killed, the kernel checks whether that process was holding the swap token and, if so, releases it; this is done by the `mmaput( )` function (see the section "[The Memory Descriptor](#)" in [Chapter 9](#)).

17.4. Swapping

Swapping has been introduced to offer a backup on disk for unmapped pages. We know from the previous discussion that there are three kinds of pages that must be handled by the swapping subsystem:

- Pages that belong to an anonymous memory region of a process (User Mode stack or heap)
- Dirty pages that belong to a private memory mapping of a process
- Pages that belong to an IPC shared memory region (see the section "[IPC Shared Memory](#)" in [Chapter 19](#))

Like demand paging, swapping must be transparent to programs. In other words, no special instruction related to swapping needs to be inserted into the code. To understand how this can be done, recall from the section "[Regular Paging](#)" in [Chapter 2](#) that each Page Table entry includes a `Present` flag. The kernel exploits this flag to signal that a page belonging to a process address space has been swapped out. Besides that flag, Linux also takes advantage of the remaining bits of the Page Table entry to store into them a "swapped-out page identifier" that encodes the location of the swapped-out page on disk. When a Page Fault exception occurs, the corresponding exception handler can detect that the page is not present in RAM and invoke the function that swaps in the missing page from disk.

The main features of the swapping subsystem can be summarized as follows:

- Set up "swap areas" on disk to store pages that do not have a disk image.
- Manage the space on swap areas allocating and freeing "page slots" as the need occurs.
- Provide functions both to "swap out" pages from RAM into a swap area and to "swap in" pages from a swap area into RAM.
- Make use of "swapped-out page identifiers" in the Page Table entries of pages that are currently swapped out to keep track of the positions of data in the swap areas.

To sum up, swapping is the crowning feature of page frame reclaiming. If we want to be sure that all the page frames obtained by a process, and not only those containing pages that have an image on disk, can be reclaimed at will by the PFRA, then swapping has to be used. Of course, you might turn off swapping by using the *swapoff* command; in this case, however, disk thrashing is likely to occur sooner when the system load increases.

We should also mention that swapping can be used to expand the memory address space that is effectively usable by the User Mode processes. In fact, large swap areas allow the kernel to launch several

demanding applications whose total memory requests exceed the amount of physical RAM installed in the system. However, simulation of RAM is not like RAM in terms of performance. Every access by a process to a page that is currently swapped out is of several orders of magnitude longer than an access to a page in RAM. In short, if performance is of great importance, swapping should be used only as a last resort; adding RAM chips still remains the best solution to cope with increasing computing needs.

17.4.1. Swap Area

The pages swapped out from memory are stored in a swap area, which may be implemented either as a disk partition of its own or as a file included in a larger partition. Several different swap areas may be defined, up to a maximum number specified by the `MAX_SWAPFILES` macro (usually set to 32).

Having multiple swap areas allows a system administrator to spread a lot of swap space among several disks so that the hardware can act on them concurrently; it also lets swap space be increased at runtime without rebooting the system.

Each swap area consists of a sequence of page slots : 4,096-byte blocks used to contain a swapped-out page. The first page slot of a swap area is used to persistently store some information about the swap area; its format is described by the `swap_header` union composed of two structures, `info` and `magic`. The `magic` structure provides a string that marks part of the disk unambiguously as a swap area; it consists of just one field, `magic.magic`, which contains a 10-character "magic" string. The `magic` structure essentially allows the kernel to unambiguously identify a file or a partition as a swap area; the text of the string, namely "SWAPSPACE2," is always located at the end of the first page slot.

The `info` structure includes the following fields:

`bootbits`

Not used by the swapping algorithm; this field corresponds to the first 1,024 bytes of the swap area, which may store partition data, disk labels, and so on.

`version`

Swapping algorithm version.

`last_page`

Last page slot that is effectively usable.

`nr_badpages`

Number of defective page slots.

padding[125]

Padding bytes.

badpages[1]

Up to 637 numbers specifying the location of defective page slots.

17.4.1.1. Creating and activating a swap area

The data stored in a swap area is meaningful as long as the system is on. When the system is switched off, all processes are killed, so the data stored by processes in swap areas is discarded. For this reason, swap areas contain very little control information: essentially, the swap area type and the list of defective page slots. This control information easily fits in a single 4 KB page.

Usually, the system administrator creates a swap partition when creating the other partitions on the Linux system, and then uses the *mkswap* command to set up the disk area as a new swap area. That command initializes the fields just described within the first page slot. Because the disk may include some bad blocks, the program also examines all other page slots to locate the defective ones. But executing the *mkswap* command leaves the swap area in an inactive state. Each swap area can be activated in a script file at system boot or dynamically after the system is running.

Each swap area consists of one or more swap extents, each of which is represented by a `swap_extent` descriptor. Each extent corresponds to a group of pages or more accurately, page slots that are physically adjacent on disk. Hence, the `swap_extent` descriptor includes the index of the first page of the extent in the swap area, the length in pages of the extent, and the starting disk sector number of the extent. An ordered list of the extents that compose a swap area is created when activating the swap area itself. A swap area stored in a disk partition is composed of just one extent; conversely, a swap area stored in a regular file can be composed of several extents, because the filesystem may not have allocated the whole file in contiguous blocks on disk.

17.4.1.2. How to distribute pages in the swap areas

When swapping out, the kernel tries to store pages in contiguous page slots to minimize disk seek time when accessing the swap area; this is an important element of an efficient swapping algorithm.

However, if more than one swap area is used, things become more complicated. Faster swap areas stored in faster disks get a higher priority. When looking for a free slot, the search starts in the swap area that has the highest priority. If there are several of them, swap areas of the same priority are cyclically selected to avoid overloading one of them. If no free slot is found in the swap areas that have the highest priority, the search continues in the swap areas that have a priority next to the highest one, and so on.

17.4.2. Swap Area Descriptor

Each active swap area has its own `swap_info_struct` descriptor in memory. The fields of the descriptor are illustrated in [Table 17-3](#).

Table 17-3. Fields of a swap area descriptor

Type	Field	Description
<code>unsigned int</code>	<code>flags</code>	Swap area flags
<code>spinlock_t</code>	<code>sdev_lock</code>	Spin lock protecting the swap area
<code>struct file *</code>	<code>swap_file</code>	Pointer to the file object of the regular file or device file that stores the swap area
<code>struct block_device *</code>	<code>bdev</code>	Descriptor of the block device containing the swap area
<code>struct list_head</code>	<code>extent_list</code>	Head of the list of extents that compose the swap area
<code>int</code>	<code>nr_extents</code>	Number of extents composing the swap area
<code>struct swap_extent *</code>	<code>curr_swap_extent</code>	Pointer to the most recently used extent descriptor
<code>unsigned int</code>	<code>old_block_size</code>	Natural block size of the partition containing the swap area
<code>unsigned short *</code>	<code>swap_map</code>	Pointer to an array of counters, one

unsigned int	lowest_bit	for each swap area page slot First page slot to be scanned when searching for a free one
unsigned int	highest_bit	Last page slot to be scanned when searching for a free one
unsigned int	cluster_next	Next page slot to be scanned when searching for a free one
unsigned int	cluster_nr	Number of free page slot allocations before restarting from the beginning
int	prio	Swap area priority
int	pages	Number of usable page slots
unsigned long	max	Size of swap area in pages
unsigned long	inuse_pages	Number of used page slots in the swap area
int	next	Pointer to next swap area descriptor

The `flags` field includes three overlapping subfields:

SWP_USED

1 if the swap area is active; 0 if it is inactive.

SWP_WRITEOK

1 if it is possible to write into the swap area; 0 if the swap area is read-only (it is being activated or inactivated).

SWP_ACTIVE

This 2-bit field is actually the combination of `SWP_USED` and `SWP_WRITEOK`; the flag is set when both the previous flags are set.

The `swap_map` field points to an array of counters, one for each swap area page slot. If the counter is equal to 0, the page slot is free; if it is positive, the page slot is filled with a swapped-out page. Essentially, the page

slot counter denotes the number of processes that share the swapped-out page. If the counter has the value `SWAP_MAP_MAX` (equal to 32,767), the page stored in the page slot is "permanent" and cannot be removed from the corresponding slot. If the counter has the value `SWAP_MAP_BAD` (equal to 32,768), the page slot is considered defective, and thus unusable.^[*]

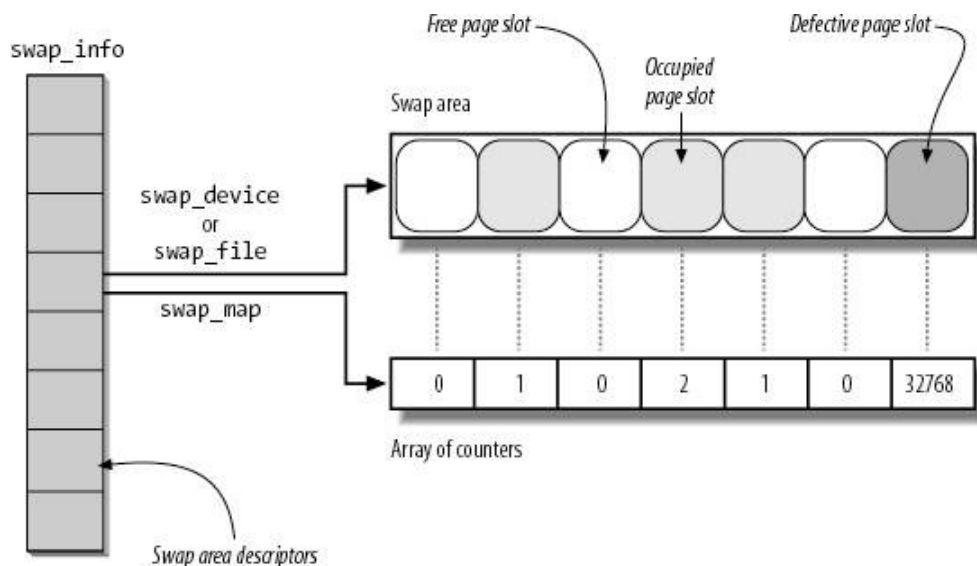
[*] "Permanent" page slots protect against overflows of `swap_map` counters. Without them, valid page slots could become "defective" if they are referenced too many times, thus leading to data losses. However, no one really expects that a page slot counter could reach the value 32,768. It's just a "belt and suspenders" approach.

The `prio` field is a signed integer that denotes the order in which the swap subsystem should consider each swap area.

The `sdev_lock` field is a spin lock that protects the swap area's data structures chiefly, the swap descriptor against concurrent accesses in SMP systems.

The `swap_info` array includes `MAX_SWAPFILES` swap area descriptors. Only the areas whose `SWP_USED` flags are set are used, because they are the activated areas. Figure 17-6 illustrates the `swap_info` array, one swap area, and the corresponding array of counters.

Figure 17-6. Swap area data structures



The `nr_swapfiles` variable stores the index of the last array element that contains, or that has contained, a used swap area descriptor. Despite its name, the variable does not contain the number of active swap areas.

Descriptors of active swap areas are also inserted into a list sorted by the swap area priority. The list is implemented through the `next` field of the swap area descriptor, which stores the index of the next descriptor in the `swap_info` array. This use of the field as an index is different from most fields with the name `next`, which are usually pointers.

The `swap_list` variable, of type `swap_list_t`, includes the following fields:

head

Index in the `swap_info` array of the first list element.

next

Index in the `swap_info` array of the descriptor of the next swap area to be selected for swapping out pages. This field is used to implement a Round Robin algorithm among maximum-priority swap areas with free slots.

The `swaplock` spin lock protects the list against concurrent accesses in multiprocessor systems.

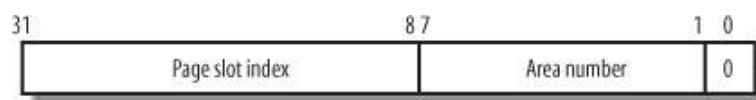
The `max` field of the swap area descriptor stores the size of the swap area in pages, while the `pages` field stores the number of usable page slots. These numbers differ because `pages` does not take the first page slot and the defective page slots into consideration.

Finally, the `nr_swap_pages` variable contains the number of available (free and nondefective) page slots in all active swap areas, while the `total_swap_pages` variable contains the total number of nondefective page slots.

17.4.3. Swapped-Out Page Identifier

A swapped-out page is uniquely identified quite simply by specifying the index of the swap area in the `swap_info` array and the page slot index inside the swap area. Because the first page (with index 0) of the swap area is reserved for the `swap_header` union discussed earlier, the first useful page slot has index 1. The format of a swapped-out page identifier is illustrated in [Figure 17-7](#).

Figure 17-7. Swapped-out page identifier



The `swp_entry(type, offset)` function constructs a swapped-out page identifier from the swap area index `type` and the page slot index `offset`. Conversely, the `swp_type` and `swp_offset` functions extract from a swapped-out page identifier the swap area index and the page slot index, respectively.

When a page is swapped out, its identifier is inserted as the page's entry into the Page Table so the page can be found again when needed. Notice that the least-significant bit of such an identifier, which corresponds to the `Present` flag, is always cleared to denote the fact that the page is not currently in RAM. However, at least one of the remaining 31 bits has to be set because no page is ever stored in slot 0 of swap area 0. It is therefore possible to identify three different cases from the value of a Page Table entry:

Null entry

The page does not belong to the process address space, or the underlying page frame has not yet been assigned to the process (demand paging).

First 31 most-significant bits not all equal to 0, last bit equal to 0
 The page is currently swapped out.

Least-significant bit equal to 1
 The page is contained in RAM.

The maximum size of a swap area is determined by the number of bits available to identify a slot. On the 80 x 86 architecture, the 24 bits available limit the size of a swap area to 2^{24} slots (that is, to 64 GB).

Because a page may belong to the address spaces of several processes (see the earlier section "[Reverse Mapping](#)"), it may be swapped out from the address space of one process and still remain in main memory; therefore, it is possible to swap out the same page several times. A page is physically swapped out and stored just once, of course, but each subsequent attempt to swap it out increases the `swap_map` counter.

The `swap_duplicate( )` function is usually invoked while trying to swap out an already swapped-out page. It simply verifies that the swapped-out page identifier passed as its parameter is valid and increases the corresponding `swap_map` counter. More precisely, it performs the following actions:

1. Uses the `swp_type` and `swp_offset` functions to extract the swap area number and the page slot index from the parameter.
2. Checks whether the swap area number identified is active; if not, it returns 0 (invalid identifier).
3. Checks whether the page slot is valid and not free (its `swap_map` counter is greater than 0 and less than `SWAP_MAP_BAD`); if not, it returns 0 (invalid identifier).
4. Otherwise, the swapped-out page identifier locates a valid page. Increases the `swap_map` counter of the page slot if it has not already reached the value `SWAP_MAP_MAX`.
5. Returns 1 (valid identifier).

17.4.4. Activating and Deactivating a Swap Area

Once a swap area is initialized, the superuser (or, more precisely, every user having the `CAP_SYS_ADMIN` capability, as described in the section "[Process Credentials and Capabilities](#)" in [Chapter 20](#)) may use the `swapon` and `swapoff` programs to activate and deactivate the swap area, respectively. These programs use the `swapon( )` and `swapoff( )` system calls; we'll briefly sketch out the corresponding service routines.

17.4.4.1. The `sys_swapon( )` service routine

The `sys_swapon( )` service routine receives the following as its parameters:

`specialfile`

This parameter points to the pathname (in the User Mode address space) of the device file (partition) or plain file used to implement the swap area.

`swap_flags`

This parameter consists of a single `SWAP_FLAG_PREFER` bit plus 31 bits of priority of the swap area (these bits are significant only if the `SWAP_FLAG_PREFER` bit is on).

The function checks the fields of the `swap_header` union that was put in the first slot when the swap area was created. The function performs these main steps:

1. Checks that the current process has the `CAP_SYS_ADMIN` capability.
2. Looks in the first `nr_swapfiles` components of the `swap_info` array of swap area descriptors for the first descriptor having the `SWP_USED` flag cleared, meaning that the corresponding swap area is inactive. If an inactive swap area is found, it goes to step 4.
3. The new swap area array index is equal to `nr_swapfiles`: it checks that the number of bits reserved for the swap area index is sufficiently large to encode the new index; if not, returns an error code; otherwise, it increases by one the value of `nr_swapfiles`.
4. An index of an unused swap area has been found: it initializes the descriptor's fields; in particular, it sets `flags` to `SWP_USED`, and sets `lowest_bit` and `highest_bit` to 0.
5. If the `swap_flags` parameter specifies a priority for the new swap area, the function sets the `prio` field of the descriptor. Otherwise, it initializes the field to one less than the lowest priority among all active swap areas (thus assuming that the last activated swap area is on the slowest block device). If no other swap areas are already active, the function assigns the value -1.
6. Copies the string pointed to by the `specialfile` parameter from the User Mode address space.
7. Invokes `filp_open( )` to open the file specified by the `specialfile` parameter (see the section "[The open\(\) System Call](#)" in [Chapter 12](#)).
8. Stores the addresses of the file object returned by `filp_open( )` in the `swap_file` field of the swap area descriptor.
9. Makes sure that the swap area is not already activated by looking at the other active swap areas in `swap_info`. This is done by checking the addresses of the `address_space` objects stored in the `swap_file->f_mapping` field of the swap area descriptors. If the swap area is already active, it returns an error code.
10. If the `specialfile` parameter identifies a block device file, it performs the following substeps:
 - a. Invokes `bd_claim( )` to set the swapping subsystem as the holder of the block device (see the section "[Block Devices](#)" in [Chapter 14](#)). If the block device already has a holder, it returns an error code.
 - b. Stores the address of the `block_device` descriptor in the `bdev` field of the swap area descriptor.
 - c. Stores the current block size of the device in the `old_block_size` field of the swap area descriptor, then sets the block size of the device to 4,096 bytes (the page size).
11. If the `specialfile` parameter identifies a regular file, it performs the following substeps:
 - a. Checks the `S_SWAPFILE` field of the `i_flags` field of the file's inode. If this flag is set, it returns an error code because the file is already being used as a swap area.
 - b. Stores the descriptor address of the block device containing the file in the `bdev` field of the swap area descriptor.
12. Reads the `swap_header` descriptor stored in slot 0 of the swap area. To that end, it invokes `read_cache_page( )` passing as parameters the `address_space` object pointed to by `swap_file->f_mapping`, the page index 0, the address of the file's `readpage` method (stored in `swap_file->f_mapping->a_ops->readpage`), and the pointer to the file object `swap_file`. Waits until the page has been read into memory.
13. Checks that the magic string in the last 10 characters of the first page is equal to "SWAPSPACE2." If not, it returns an error code.
14. Initializes the `lowest_bit` and `highest_bit` fields of the swap area descriptor according to the size of the swap area stored in the `info.last_page` field of the `swap_header` union.
15. Invokes `vmalloc( )` to create the array of counters associated with the new swap area and stores its address in the `swap_map` field of the swap descriptor. Initializes the elements of the array to 0 or to `SWAP_MAP_BAD`, according to the list of defective page slots stored in the `info.bad_pages` field of the `swap_header` union.
16. Computes the number of useful page slots by accessing the `info.last_page` and

- `info.nr_badpages` fields in the first page slot, and stores it in the `pages` field of the swap area descriptor. Also sets the `max` field with the total number of pages in the swap area.
- 17. Builds the `extent_list` list of swap extents for the new swap area (only one if the swap area is a disk partition), and sets properly the `nr_extents` and `curr_swap_extent` fields in the swap area descriptor.
- 18. Sets the `flags` field of the swap area descriptor to `SWP_ACTIVE`.
- 19. Updates the `nr_good_pages`, `nr_swap_pages`, and `total_swap_pages` global variables.
- 20. Inserts the swap area descriptor in the list to which the `swap_list` variable points.
- 21. Returns 0 (success).

17.4.4.2. The `sys_swapoff( )` service routine

The `sys_swapoff( )` service routine deactivates a swap area identified by the parameter `specialfile`. It is much more complex and time-consuming than `sys_swapon( )`, since the partition to be deactivated might still contain pages that belong to several processes. The function is thus forced to scan the swap area and to swap in all existing pages. Because each swap-in requires a new page frame, it might fail if there are no free page frames left. In this case, the function returns an error code. All this is achieved by performing the following major steps:

1. Checks that the current process has the `CAP_SYS_ADMIN` capability.
2. Copies the string pointed to by the `specialfile` parameter in kernel space.
3. Invokes `filp_open( )` to open the file referenced by the `specialfile` parameter; as usual, this function returns the address of a file object.
4. Scans the `swap_list` list of the swap area descriptor, and compares the address of the file object returned by `filp_open( )` with the addresses stored in the `swap_file` fields of the active swap area descriptors. If no match is found, an invalid parameter was passed to the function, so it returns an error code.
5. Invokes `cap_vm_enough_memory( )` to check whether there are enough free page frames to swap in all pages stored in the swap area. If not, the swap area cannot be deactivated; it releases the file object and returns an error code. This is only a rough check, but it could save the kernel from a lot of useless disk activity. While performing this check, `cap_vm_enough_memory( )` takes into account the page frames allocated through slab caches having the `SLAB_RECLAIM_ACCOUNT` flag set (see the section "[Interfacing the Slab Allocator with the Zoned Page Frame Allocator](#)" in [Chapter 8](#)). The number of such pages, which are considered as reclaimable, is stored in the `slab_reclaim_pages` variable.
6. Removes the swap area descriptor from the `swap_list` list.
7. Updates the `nr_swap_pages` and `total_swap_pages` variables by subtracting the value in the `pages` field of the swap area descriptor.
8. Clears the `SWP_WRITEOK` flag in the `flags` field of the swap area descriptor; this forbids the PFRA from swapping out more pages in the swap area.
9. Invokes `try_to_unuse( )` (see below) to successively force all pages left in the swap area into RAM and to correspondingly update the Page Tables of the processes that use these pages. While executing this function, the current process, which is executing the `swapoff` command, has the `PF_SWAPOFF` flag set. Setting this flag has just one consequence: in case of a dramatic shortage of page frames, the `select_bad_process( )` function will be forced to select and kill this process! (See the section "[The Out of Memory Killer](#)" earlier in this chapter.)
10. Waits until the block device driver that contains the swap area is unplugged (see the section "[Activating the Block Device Driver](#)" in [Chapter 14](#)). In this way, the reading requests submitted by `try_to_unuse( )` will be handled by the driver before the swap area is deactivated.
11. If `try_to_unuse( )` fails in allocating all requested page frames, the swap area cannot be deactivated. Therefore, the function executes the following substeps:

- a. Reinserts the swap area descriptor in the `swap_list` list and sets its `flags` field to `SWP_WRITEOK`.
 - b. Restores the original contents of the `nr_swap_pages` and `total_swap_pages` variables by adding the value in the `pages` field of the swap area descriptor.
 - c. Invokes `filp_close( )` to close the file opened in step 3 (see the section "[The close\(\) System Call](#)" in [Chapter 12](#)), and returns an error code.
12. Otherwise, all used page slots have been successfully transferred to RAM. Therefore, the function executes the following substeps:
- a. Releases the memory areas used to store the `swap_map` array and the extent descriptors.
 - b. If the swap area is stored in a disk partition, it restores the block size to its original value, which is stored in the `old_block_size` field of the swap area descriptor; moreover, it invokes the `bd_release( )` function so that the swap subsystem no longer holds the block device (see step 10a in the description of `sys_swapon( )`).
 - c. If the swap area is stored in a regular file, it clears the `S_SWAPFILE` flag of the file's inode.
 - d. Invokes `filp_close( )` twice, the first time on the `swap_file` file object, the second time on the object returned by `filp_open( )` in step 3.
 - e. Returns 0 (success).

17.4.4.3. The `try_to_unuse( )` function

The `TRy_to_unuse( )` function acts on an index parameter that identifies the swap area to be emptied; it swaps in pages and updates all the Page Tables of processes that have swapped out pages in this swap area. To that end, the function visits the address spaces of all kernel threads and processes, starting with the `init_mm` memory descriptor that is used as a marker. It is a time-consuming function that runs mostly with the interrupts enabled. Synchronization with other processes is therefore critical.

The `TRy_to_unuse( )` function scans the `swap_map` array of the swap area. When the function finds a in-use page slot, it first swaps in the page, and then starts looking for the processes that reference the page. The ordering of these two operations is crucial to avoid race conditions. While the I/O data transfer is ongoing, the page is locked, so no process can access it. Once the I/O data transfer completes, the page is locked again by `try_to_unuse( )`, so it cannot be swapped out again by another kernel control path. Race conditions are also avoided because each process looks up the page cache before starting a swap-in or swap-out operation (see the later section "[The Swap Cache](#)"). Finally, the swap area considered by `try_to_unuse( )` is marked as nonwritable (`SWP_WRITEOK` flag is not set), so no process can perform a swap-out on a page slot of this area.

However, `try_to_unuse( )` might be forced to scan the `swap_map` array of usage counters of the swap area several times. This is because memory regions that contain references to swapped-out pages might disappear during one scan and later reappear in the process lists.

For instance, recall the description of the `do_munmap( )` function (in the section "[Releasing a Linear Address Interval](#)" in [Chapter 9](#)): whenever a process releases an interval of linear addresses, `do_munmap( )` removes from the process list all memory regions that include the affected linear addresses; later, the function reinserts the memory regions that have been only partially unmapped in the process list. `do_munmap( )` takes care of freeing the swapped-out pages that belong to the interval of released linear addresses. It commendably doesn't free the swapped-out pages that belong to the memory regions that have to be reinserted in the process list.

Hence, `TRy_to_unuse( )` might fail in finding a process that references a given page slot because the corresponding memory region is temporarily not included in the process list. To cope with this fact, `try_to_unuse( )` keeps scanning the `swap_map` array until all reference counters are null. Eventually,

the ghost memory regions referencing the swapped-out pages will reappear in the process lists, so `TRy_to_unuse( )` will succeed in freeing all page slots.

Let's describe now the major operations executed by `TRy_to_unuse( )`. It executes a continuous loop on the reference counters in the `swap_map` array of the swap area passed as its parameter. This loop is interrupted and the function returns an error code if the current process receives a signal. For each reference counter, the function performs the following steps:

1. If the counter is equal to 0 (no page is stored there) or to `SWAP_MAP_BAD`, it continues with the next page slot.
2. Otherwise, it invokes the `read_swap_cache_async( )` function (see the section "[Swapping in Pages](#)" later in this chapter) to swap in the page. This consists of allocating, if necessary, a new page frame, filling it with the data stored in the page slot, and putting the page in the swap cache.
3. Waits until the new page has been properly updated from disk and locks it.
4. While the function was executing the previous step, the process could have been suspended. Therefore, it checks again whether the reference counter of the page slot is null; if so, this swap page has been freed by another kernel control path, so the function continues with the next page slot.
5. Invokes `unuse_process( )` on every memory descriptor in the doubly linked list whose head is `init_mm` (see the section "[The Memory Descriptor](#)" in [Chapter 9](#)). This time-consuming function scans all Page Table entries of the process that owns the memory descriptor, and replaces each occurrence of the swapped-out page identifier with the physical address of the page frame. To reflect this move, the function also decreases the page slot counter in the `swap_map` array (unless it is equal to `SWAP_MAP_MAX`) and increases the usage counter of the page frame.
6. Invokes `shmem_unuse( )` to check whether the swapped-out page is used as an IPC shared memory resource and to properly handle that case (see the section "[IPC Shared Memory](#)" in [Chapter 19](#)).
7. Checks the value of the reference counter of the page. If it is equal to `SWAP_MAP_MAX`, the page slot is "permanent." To free it, it forces the value 1 into the reference counter.
8. The swap cache might own the page as well (it contributes to the value of the reference counter). If the page belongs to the swap cache, it invokes the `swap_writepage( )` function to flush its contents to disk (if the page is dirty) and invokes `delete_from_swap_cache( )` to remove the page from the swap cache and to decrease its reference counter.
9. Sets the `PG_dirty` flag of the page descriptor, unlocks the page frame, and decreases its reference counter (to undo the increment done in step 5).
10. Checks the `need_resched` field of the current process; if it is set, it invokes `schedule( )` to relinquish the CPU. Deactivating a swap area is a long job, and the kernel must ensure that the other processes in the system still continue to execute. The `try_to_unuse( )` function continues from this step whenever the process is selected again by the scheduler.
11. Proceeds with the next page slot, starting at step 1.

The function continues until every reference counter in the `swap_map` array is null. Recall that even if the function starts examining the next page slot, the reference counter of the previous page slot could still be positive. In fact, a "ghost" process could still reference the page, typically because some memory regions have been temporarily removed from the process list scanned in step 5. Eventually, `try_to_unuse( )` catches every reference. In the meantime, however, the page is no longer in the swap cache, it is unlocked, and a copy is still included in the page slot of the swap area being deactivated.

One might expect that this situation could lead to data loss. For instance, suppose that some "ghost" process accesses the page slot and starts swapping the page in. Because the page is no longer in the swap cache, the process fills a new page frame with the data read from disk. However, this page frame would be different from the page frames owned by the processes that are supposed to share the page with the "ghost" process.

This problem does not arise when deactivating a swap area, because interference from a ghost process could happen only if a swapped-out page belongs to a private anonymous memory mapping.^[*] In this case, the page

frame is handled by means of the Copy On Write mechanism described in [Chapter 9](#), so it is perfectly legal to assign different page frames to the processes that reference the page. However, the `try_to_unuse( )` function marks the page as "dirty" (step 9); otherwise, the `shrink_list( )` function might later drop the page from the Page Table of some process without saving it in another swap area (see the later section "[Swapping Out Pages](#)").

[*] Actually, the page might also belong to an IPC shared memory region; [Chapter 19](#) has a discussion of this case.

17.4.5. Allocating and Releasing a Page Slot

As we will see later, when freeing memory, the kernel swaps out many pages in a short period of time. It is therefore important to try to store these pages in contiguous slots to minimize disk seek time when accessing the swap area.

A first approach to an algorithm that searches for a free slot could choose one of two simplistic, rather extreme strategies:

- Always start from the beginning of the swap area. This approach may increase the average seek time during swap-out operations, because free page slots may be scattered far away from one another.
- Always start from the last allocated page slot. This approach increases the average seek time during swap-in operations if the swap area is mostly free (as is usually the case), because the handful of occupied page slots may be scattered far away from one another.

Linux adopts a hybrid approach. It always starts from the last allocated page slot unless one of these conditions occurs:

- The end of the swap area is reached.
- `SWAPFILE_CLUSTER` (usually 256) free page slots were allocated after the last restart from the beginning of the swap area.

The `cluster_nr` field in the `swap_info_struct` descriptor stores the number of free page slots allocated. This field is reset to 0 when the function restarts allocation from the beginning of the swap area. The `cluster_next` field stores the index of the first page slot to be examined in the next allocation.^[*]

[*] As you may have noticed, the names of Linux data structures are not always appropriate. In this case, the kernel does not really "cluster" page slots of a swap area.

To speed up the search for free page slots, the kernel keeps the `lowest_bit` and `highest_bit` fields of each swap area descriptor up-to-date. These fields specify the first and the last page slots that could be free; in other words, every page slot below `lowest_bit` and above `highest_bit` is known to be occupied.

17.4.5.1. The `scan_swap_map( )` function

The `scan_swap_map( )` function is used to find a free page slot in a given swap area. It acts on a single parameter, which points to a swap area descriptor and returns the index of a free page slot. It returns 0 if the swap area does not contain any free slots. The function performs the following steps:

1. It tries first to use the current cluster. If the `cluster_nr` field of the swap area descriptor is positive, it scans the `swap_map` array of counters starting from the element at index `cluster_next` and looks for a null entry. If a null entry is found, it decreases the `cluster_nr`

- field and goes to step 4.
2. If this point is reached, either the `cluster_nr` field is null or the search starting from `cluster_next` didn't find a null entry in the `swap_map` array. It is time to try the second stage of the hybrid search. The function reinitializes `cluster_nr` to `SWAPFILE_CLUSTER` and restarts scanning the array from the `lowest_bit` index trying to find a group of `SWAPFILE_CLUSTER` free page slots. If such a group is found, it goes to step 4.
 3. No group of `SWAPFILE_CLUSTER` free page slots exists. The function restarts scanning the array from the `lowest_bit` index trying to find a single free page slot. If no null entry is found, it sets the `lowest_bit` field to the maximum index in the array, the `highest_bit` field to 0, and returns 0 (the swap area is full).
 4. A null entry is found. Puts the value 1 in the entry, decreases `nr_swap_pages`, updates the `lowest_bit` and `highest_bit` fields if necessary, increases the `inuse_pages` field by one, and sets the `cluster_next` field to the index of the page slot just allocated plus 1.
 5. Returns the index of the allocated page slot.

17.4.5.2. The `get_swap_page( )` function

The `get_swap_page( )` function is used to find a free page slot by searching all the active swap areas. The function, which returns the swapped-out page identifier of a newly allocated page slot or 0 if all swap areas are filled, takes into consideration the different priorities of the active swap areas.

Two passes are done in order to minimize runtime when it's easy to find a page slot. The first pass is partial and applies only to areas that have a single priority; the function searches such areas in a Round Robin fashion for a free slot. If no free page slot is found, a second pass is made starting from the beginning of the swap area list; during this second pass, all swap areas are examined. More precisely, the function performs the following steps:

1. If `nr_swap_pages` is null or if there are no active swap areas, it returns 0.
2. Starts by considering the swap area pointed to by `swap_list.next` (recall that the swap area list is sorted by decreasing priorities).
3. If the swap area is active, it invokes `scan_swap_map( )` to allocate a free page slot. If `scan_swap_map( )` returns a page slot index, the function's job is essentially done, but it must prepare for its next invocation. Thus, it updates `swap_list.next` to point to the next swap area in the swap area list, if the latter has the same priority (thus continuing the round-robin use of these swap areas). If the next swap area does not have the same priority as the current one, the function sets `swap_list.next` to the first swap area in the list (so that the next search will start with the swap areas that have the highest priority). The function finishes by returning the swapped-out page identifier corresponding to the page slot just allocated.
4. Either the swap area is not writable, or it does not have free page slots. If the next swap area in the swap area list has the same priority as the current one, the function makes it the current one and goes to step 3.
5. At this point, the next swap area in the swap area list has a lower priority than the previous one. The next step depends on which of the two passes the function is performing.
 - a. If this is the first (partial) pass, it considers the first swap area in the list and goes to step 3, thus starting the second pass.
 - b. Otherwise, it checks if there is a next element in the list; if so, it considers it and goes to step 3.
6. At this point the list is completely scanned by the second pass and no free page slot has been found; it returns 0.

17.4.5.3. The `swap_free( )` function

The `swap_free( )` function is invoked when swapping in a page to decrease the corresponding `swap_map` counter (see [Table 17-3](#)). When the counter reaches 0, the page slot becomes free since its identifier is no longer included in any Page Table entry. We'll see in the later section "[The Swap Cache](#)," however, that the swap cache counts as an owner of the page slot.

The function acts on a single `entry` parameter that specifies a swapped-out page identifier and performs the following steps:

1. Derives the swap area index and the `offset` page slot index from the `entry` parameter and gets the address of the swap area descriptor.
2. Checks whether the swap area is active and returns right away if it is not.
3. If the `swap_map` counter corresponding to the page slot being freed is smaller than `SWAP_MAP_MAX`, the function decreases it. Recall that entries that have the `SWAP_MAP_MAX` value are considered persistent (undeletable).
4. If the `swap_map` counter becomes 0, the function increases the value of `nr_swap_pages`, decreases the `inuse_pages` field, and updates, if necessary, the `lowest_bit` and `highest_bit` fields of the swap area descriptor.

17.4.6. The Swap Cache

Transferring pages to and from a swap area is an activity that can induce many race conditions. In particular, the swapping subsystem must handle carefully the following cases:

Multiple swap-ins

Two processes may concurrently try to swap in the same shared anonymous page.

Concurrent swap-ins and swap-outs

A process may swap-in a page that is being swapped out by the PFRA.

The swap cache has been introduced to solve these kinds of synchronization problems. The key rule is that nobody can start a swap-in or swap-out without checking whether the swap cache already includes the affected page. Thanks to the swap cache, concurrent swap operations affecting the same page always act on the same page frame; therefore, the kernel may safely rely on the `PG_locked` flag of the page descriptor to avoid any race condition.

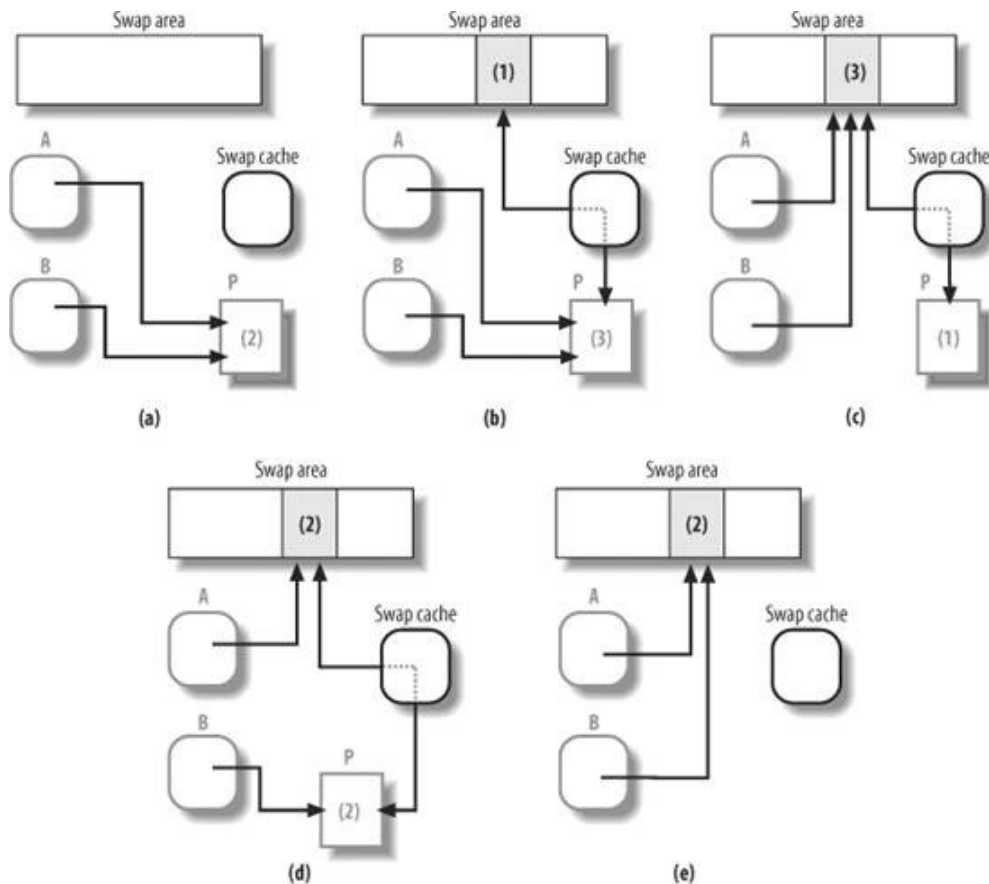
For example, consider two processes that share the same swapped-out page. When the first process tries to access the page, the kernel starts the swap-in operation. The very first step consists of checking whether the page frame is already included in the swap cache. Let's suppose it isn't: then, the kernel allocates a new page frame and inserts it into the swap cache; next, it starts the I/O operation to read the page's contents from the swap area. Meanwhile, the second process accesses the shared anonymous page. As above, the kernel starts a swap-in operation and checks whether the affected page frame is already included in the swap cache. Now, it is included, thus the kernel simply accesses the page frame descriptor and puts the current process to sleep until the `PG_locked` flag is cleared, that is, until the I/O data transfer completes.

The swap cache plays a crucial role also when concurrent swap-in and swap-out operations mix up. As explained in the section "[Low On Memory Reclaiming](#)" earlier in this chapter, the `shrink_list( )` function starts swapping out an anonymous page only if `TRY_to_unmap( )` succeeds in removing the page frame from the User Mode Page Tables of all processes that own the page. However, one of these processes

may access the page and cause a swap-in while the swap-out write operation is still in progress.

Before being written to disk, each page to be swapped out is stored in the swap cache by `shrink_list()`. Consider a page P that is shared among two processes, A and B. Initially, the Page Table entries of both processes contain a reference to the page frame, and the page has two owners; this case is illustrated in Figure 17-8(a). When the PFRA selects the page for reclaiming, `shrink_list()` inserts the page frame in the swap cache. As illustrated in Figure 17-8(b), now the page frame has three owners, while the page slot in the swap area is referenced only by the swap cache. Next, the PFRA invokes `try_to_unmap()` to remove the references to the page frame from the Page Table of the processes; once this function terminates, the page frame is referenced only by the swap cache, while the page slot is referenced by the two processes and the swap cache, as illustrated in Figure 17-8(c). Let's suppose that, while the page's contents are being written to disk, process B accesses the page that is, it tries to access a memory cell using a linear address inside the page. Then, the page fault handler finds the page frame in the swap cache and puts back its physical address in the Page Table entry of process B, as illustrated in Figure 17-8(d). Conversely, if the swap-out operation terminates without concurrent swap-in operations, the `shrink_list()` function removes the page frame from the swap cache and releases the page frame to the Buddy system, as illustrated in Figure 17-8(e).

Figure 17-8. The role of the swap cache



You might consider the swap cache as a transit area containing the page descriptors of anonymous pages that are being currently swapped-in or swapped out. When the swap-in or swap-out terminates (in the case of shared anonymous pages, the swap-in or swap-out must have been performed on all the processes that share the page), the page descriptor of the anonymous page may be removed from the swap cache.^[*]

[*] In some cases, the swap cache improves also the system performance: consider a server daemon that services requests by creating child processes. Under heavy system load, a page can get swapped out from the parent process, and it will never be paged in for the parent process. Without the swap cache, every child process that gets forked off needs to fault that page in from the swap area.

17.4.6.1. Swap cache implementation

The swap cache is implemented by the page cache data structures and procedures, which are described in the section "The Page Cache" in Chapter 15. Recall that the core of the page cache is a set of radix trees that allows the algorithm to quickly derive the address of a page descriptor from the address of an `address_space` object identifying the owner of the page as well as from an offset value.

Pages in the swap cache are stored as every other page in the page cache, with the following special treatment:

- The `mapping` field of the page descriptor is set to `NULL`.
- The `PG_swapcache` flag of the page descriptor is set.
- The `private` field stores the swapped-out page identifier associated with the page.

Moreover, when the page is put in the swap cache, both the `count` field of the page descriptor and the page slot usage counters are increased, because the swap cache uses both the page frame and the page slot.

Finally, a single `swapper_space` address space is used for all pages in the swap cache, so a single radix tree pointed to by `swapper_space.page_tree` addresses the pages in the swap cache. The `nrpages` field of the `swapper_space` address space stores the number of pages contained in the swap cache.

17.4.6.2. Swap cache helper functions

The kernel uses several functions to handle the swap cache; they are based mainly on those discussed in the section "The Page Cache" in Chapter 15. We show later how these relatively low-level functions are invoked by higher-level functions to swap pages in and out as needed.

The main functions that handle the swap cache are:

`lookup_swap_cache()`

Finds a page in the swap cache through its swapped-out page identifier passed as a parameter and returns the page descriptor address. It returns 0 if the page is not present in the cache. To find the required page, it invokes `radix_tree_lookup()`, passing as parameters a pointer to `swapper_space.page_tree` the radix tree used for pages in the swap cache and the swapped-out page identifier.

`add_to_swap_cache()`

Inserts a page into the swap cache. It essentially invokes `swap_duplicate()` to check whether the page slot passed as a parameter is valid and to increase the page slot usage counter; then, it invokes `radix_tree_insert()` to insert the page into the cache; finally, it increases the page's reference counter and sets the `PG_swapcache` and `PG_locked` flags.

`__add_to_swap_cache()`

Similar to `add_to_swap_cache()`, except that the function does not invoke `swap_duplicate()` before inserting the page frame in the swap cache.

`delete_from_swap_cache()`

Removes a page from the swap cache by invoking `radix_tree_delete()`, decreases the corresponding usage counter in `swap_map`, and decreases the page reference counter.

`free_page_and_swap_cache()`

Removes a page from the swap cache if no User Mode process besides `current` is referencing the corresponding page slot, and decreases the page's usage counter.

`free_pages_and_swap_cache()`

Analogous to `free_page_and_swap_cache()`, but operates on a set of pages.

`free_swap_and_cache()`

Frees a swap entry, and checks whether the page referenced by the entry is in the swap cache. If either no User Mode process, besides `current`, is referencing the page or more than 50% of the swap entries are busy, the function removes the page from the swap cache.

17.4.7. Swapping Out Pages

We have seen in the section "[Low On Memory Reclaiming](#)" earlier in this chapter how the PFRA determines whether a given anonymous page should be swapped out. In this section we show how the kernel performs a swap-out.

17.4.7.1. Inserting the page frame in the swap cache

The first step of a swap-out operation consists of preparing the swap cache. If the `shrink_list()` function determines that a page is anonymous (the `PageAnon()` function returns 1) and that the swap cache does not include the corresponding page frame (the `PG_swapcache` flag in the page descriptor is clear), the kernel invokes the `add_to_swap()` function.

The `add_to_swap()` function allocates a new page slot in a swap area and inserts a page frame whose page descriptor address is passed as its parameter in the swap cache. Essentially, the function performs the following steps:

1. Invokes `get_swap_page()` to allocate a new page slot; see the section "[Allocating and Releasing a Page Slot](#)" earlier in this chapter. Returns 0 in case of failure (for example, no free page slot found).
2. Invokes `__add_to_page_cache()`, passing to it the page slot index, the page descriptor address, and some allocation flags.
3. Sets the `PG_uptodate` and `PG_dirty` flags in the page descriptor, so that the `shrink_list()` function will be forced to write the page to disk (see the next section).
4. Returns 1 (success).

17.4.7.2. Updating the Page Table entries

Once `add_to_swap()` terminates, `shrink_list()` invokes `try_to_unmap()`, which determines the address of every User Mode page table entry referring to the anonymous page and writes into it a swapped-out page identifier; this is described in the section "[Reverse Mapping for Anonymous Pages](#)" earlier in this chapter.

17.4.7.3. Writing the page into the swap area

The next action to be performed to complete the swap-out consists of writing the page's contents into the swap area. This I/O transfer is activated by the `shrink_list()` function, which checks whether the `PG_dirty` flag of the page frame is set and consequently executes the `pageout()` function (see [Figure 17-5](#) earlier in this chapter).

As explained in the section "[Low On Memory Reclaiming](#)" earlier in this chapter, the `pageout()` function sets up a `writeback_control` descriptor and invokes the `writepage` method of the page's `address_space` object. The `writepage` method of the `swapper_state` object is implemented by the `swap_writepage()` function.

The `swap_writepage()` function, in turn, performs essentially the following steps:

1. Checks whether at least one User Mode process is referencing the page. If not, it removes the page from the swap cache and returns 0. This check is necessary because a process might race with the PRFA and release a page after the check performed by `shrink_list()`.
2. Invokes `get_swap_bio()` to allocate and initialize a `bio` descriptor (see the section "[The Bio Structure](#)" in [Chapter 14](#)). The function derives the address of the swap area descriptor from the swapped-out page identifier; then, it walks the swap extent lists to determine the initial disk sector of the page slot. The `bio` descriptor will include a request for a single page of data (the page slot); the completion method is set to the `end_swap_bio_write()` function.
3. Sets the `PG_writeback` flag in the page descriptor and the writeback tags in the swap cache's radix tree (see the section "[The Tags of the Radix Tree](#)" in [Chapter 15](#)). Moreover, the function resets the `PG_locked` flag.
4. Invokes `submit_bio()`, passing to it the `WRITE` command and the `bio` descriptor address.
5. Returns 0.

Once the I/O data transfer terminates, the `end_swap_bio_write()` function is executed. Essentially, this function wakes up any process waiting until the `PG_writeback` flag of the page is cleared, clears the `PG_writeback` flag and the corresponding tags in the radix tree, and releases the `bio` descriptor used for the I/O transfer.

17.4.7.4. Removing the page frame from the swap cache

The last step of the swap-out operation is performed once more by `shrink_list()`: if it verifies that no process has tried to access the page frame while doing the I/O data transfer, it essentially invokes `delete_from_swap_cache()` to remove the page frame from the swap cache. Because the swap cache was the only owner of the page, the page frame is released to the buddy system.

17.4.8. Swapping in Pages

Swap-in takes place when a process attempts to address a page that has been swapped out to disk. The Page Fault exception handler triggers a swap-in operation when the following conditions occur (see the section "[Handling a Faulty Address Inside the Address Space](#)" in [Chapter 9](#)):

- The page including the address that caused the exception is a valid one that is, it belongs to a memory region of the current process.
- The page is not present in memory that is, the `Present` flag in the Page Table entry is cleared.
- The Page Table entry associated with the page is not null, but the `Dirty` bit is clear; this means that the entry contains a swapped-out page identifier (see the section "[Demand Paging](#)" in [Chapter 9](#)).

If all the above conditions are satisfied, `handle_pte_fault( )` invokes a quite handy `do_swap_page( )` function to swap in the page required.

17.4.8.1. The `do_swap_page( )` function

The `do_swap_page( )` function acts on the following parameters:

`mm`
Memory descriptor address of the process that caused the Page Fault exception

`vma`
Memory region descriptor address of the region that includes `address`

`address`
Linear address that causes the exception

`page_table`
Address of the Page Table entry that maps `address`

`pmd`
Address of the Page Middle Directory that maps `address`

`orig_pte`
Content of the Page Table entry that maps `address`

`write_access`
Flag denoting whether the attempted access was a read or a write

Contrary to other functions, `do_swap_page( )` never returns 0. It returns 1 if the page is already in the swap cache (minor fault), 2 if the page was read from the swap area (major fault), and -1 if an error occurred

while performing the swap-in. It essentially executes the following steps:

1. Gets the swapped-out page identifier from `orig_pte`.
2. Invokes `pte_unmap( )` to release any temporary kernel mapping for the Page Table created by the `handle_mm_fault( )` function (see the section "[Handling a Faulty Address Inside the Address Space](#)" in [Chapter 9](#)). As explained in the section "[Kernel Mappings of High-Memory Page Frames](#)" in [Chapter 8](#), a kernel mapping is required to access a page table in high memory.
3. Releases the `page_table_lock` spin lock of the memory descriptor (it was acquired by the caller function `handle_pte_fault( )`).
4. Invokes `lookup_swap_cache( )` to check whether the swap cache already contains a page corresponding to the swapped-out page identifier; if the page is already in the swap cache, it jumps to step 6.
5. Invokes the `swpin_readahead( )` function to read from the swap area a group of at most `2n` pages, including the requested one. The value `n` is stored in the `page_cluster` variable, and is usually equal to 3.^[*] Each page is read by invoking the `read_swap_cache_async( )` function (see below).

[*] The system administrator may tune this value by writing into the `/proc/sys/vm/page-cluster` file. Swap-in read-ahead can be disabled by setting `page_cluster` to 0.

6. Invokes `read_swap_cache_async( )` once more to swap in precisely the page accessed by the process that caused the Page Fault. This step might appear redundant, but it isn't really. The `swpin_readahead( )` function might fail in reading the requested page for instance, because `page_cluster` is set to 0 or the function tried to read a group of pages including a free page slot or a defective page slot (`SWAP_MAP_BAD`). On the other hand, if `swpin_readahead( )` succeeded, this invocation of `read_swap_cache_async( )` terminates quickly because it finds the page in the swap cache.
7. If, despite all efforts, the requested page was not added to the swap cache, another kernel control path might have already swapped in the requested page on behalf of a clone of this process. This case is checked by temporarily acquiring the `page_table_lock` spin lock and comparing the entry to which `page_table` points with `orig_pte`. If they differ, the page has already been swapped in by some other kernel control path, so the function returns 1 (minor fault); otherwise, it returns -1 (failure).
8. At this point, we know that the page is in the swap cache. If the page has been effectively swapped in (major fault), the function invokes `grab_swap_token( )` to try to grab the swap token (see the section "[The Swap Token](#)" earlier in this chapter).
9. Invokes `mark_page_accessed( )` (see the earlier section "[The Least Recently Used \(LRU\) Lists](#)") and locks the page.
10. Acquires the `page_table_lock` spin lock.
11. Checks whether another kernel control path has swapped in the requested page on behalf of a clone of this process. In this case, it releases the `page_table_lock` spin lock, unlocks the page, and returns 1 (minor fault).
12. Invokes `swap_free( )` to decrease the usage counter of the page slot corresponding to `enTty`.
13. Checks whether the swap cache is at least 50 percent full (`nr_swap_pages` is smaller than half of `total_swap_pages`). If so, it checks whether the page is owned only by the process that caused the fault (or one of its clones); if this is the case, removes the page from the swap cache.
14. Increases the `rss` field of the process's memory descriptor.
15. Updates the Page Table entry so the process can find the page. The function accomplishes this by writing the physical address of the requested page and the protection bits found in the `vm_page_prot` field of the memory region into the Page Table entry addressed by `page_table`. Moreover, if the access that caused the fault was a write and the faulting process is the unique owner of the page, the function also sets the `Dirty` flag and the `Read/Write` flag to prevent a useless Copy On Write fault.
16. Unlocks the page.

17. Invokes `page_add_anon_rmap( )` to insert the anonymous page in the object-based reverse mapping data structures (see the section "[Reverse Mapping for Anonymous Pages](#)" earlier in this chapter.)
18. If the `write_access` parameter is equal to 1, the function invokes `do_wp_page( )` to make a copy of the page frame (see the section "[Copy On Write](#)" in [Chapter 9](#)).
19. Releases the `mm->page_table_lock` spin lock and returns the `ret` return code: 1 (minor fault) or 2 (major fault).

17.4.8.2. The `read_swap_cache_async( )` function

The `read_swap_cache_async( )` function is invoked whenever the kernel must swap in a page. It acts on three parameters:

entry	A swapped-out page identifier
vma	A pointer to the memory region that should contain the page
addr	The linear address of the page

As we know, before accessing the swap partition, the function must check whether the swap cache already includes the desired page frame. Therefore, the function essentially executes the following operations:

1. Invokes `radix_tree_lookup( )` to locate in the radix tree of the `swapper_space` object a page frame at the position given by the swapped-out page identifier `entry`. If the page is found, it increases its reference counter and returns the address of its descriptor.
2. The page is not included in the swap cache. Invokes `alloc_pages( )` to allocate a new page frame. If no free page frame is available, it returns 0 (indicating the system is out of memory).
3. Invokes `add_to_swap_cache( )` to insert the page descriptor of the new page frame into the swap cache. As mentioned in the earlier section "[Swap cache helper functions](#)," this function also locks the page.
4. The previous step might fail if `add_to_swap_cache( )` finds a duplicate of the page in the swap cache. For instance, the process could block in step 2, thus allowing another process to start a swap-in operation on the same page slot. In this case, it releases the page frame allocated in step 2 and restarts from step 1.
5. Invokes `lru_cache_add_active( )` to insert the page in the LRU active list (see the section "[The Least Recently Used \(LRU\) Lists](#)" earlier in this chapter).
6. The page descriptor of the new page frame is now in the swap cache. Invokes `swap_readpage( )` to read the page's contents from the swap area. This function is quite similar to `swap_writepage( )` described in the earlier section "[Swapping Out Pages](#):" it clears the `PG_uptodate` flag of the page descriptor, invokes `get_swap_bio( )` to allocate and initialize a `bio` descriptor for the I/O transfer, and invokes `submit_bio( )` to submit the I/O request to the block subsystem layer.
7. Returns the address of the page descriptor.